

Sicurezza Informatica

Appunti del Corso

Alessia Cassetta

13 aprile 2026

Indice

1	Concetti di Sicurezza Informatica	1
1.1	Livelli di Impatto (FIPS 199) ed Esempi	2
1.1.1	Le Sfide della Sicurezza Informatica	2
1.2	Un Modello per la Sicurezza Informatica	3
1.3	Minacce, Attacchi e Asset	4
1.3.1	Conseguenze delle Minacce e Azioni (Attacchi)	4
1.3.2	Minacce per Categoria di Asset	4
1.4	Requisiti Funzionali di Sicurezza	5
1.5	Principi Fondamentali di Progettazione della Sicurezza	5
1.6	Superfici di Attacco e Alberi di Attacco	6
1.6.1	Superfici di Attacco	6
1.6.2	Alberi di Attacco (Attack Trees)	6
1.7	Strategia di Sicurezza Informatica	7
2	Crittografia Simmetrica e Confidenzialità	8
2.1	Principi Fondamentali e Attacchi	8
2.2	Cifrari a Blocchi e Algoritmi Principali	9
2.3	Modalità di Operazione per Cifrari a Blocchi	10
2.4	Cifrari a Flusso e RC4	11
2.5	Posizionamento della Sicurezza e Distribuzione delle Chiavi	12
3	Autenticazione dei Messaggi e Funzioni Hash	12
3.1	Principi di Autenticazione e MAC	12
3.2	Funzioni Hash Sicure	13
4	Crittografia a Chiave Pubblica e Firme Digitali	16
4.1	Principi e Requisiti dell'Asimmetrica	16
4.2	L'algoritmo RSA	17
4.3	Scambio di Chiavi Diffie-Hellman	17
4.4	Firme Digitali e Certificati	18
4.5	Altri Algoritmi Asimmetrici	18
5	Strumenti Aggiuntivi e Applicazioni	19
5.1	Busta Digitale (Digital Envelope)	19
5.2	Numeri Casuali e Pseudocasuali	19
5.3	Protezione dei Dati Archiviati (Data at Rest)	19
6	Autenticazione dell'Utente	20
6.1	Principi dell'Autenticazione Elettronica dell'Utente	20
6.2	Autenticazione Basata su Password	21
6.3	Autenticazione Basata su Token	23
6.4	Autenticazione Biometrica	24
6.5	Autenticazione Utente Remota	25

6.6	Problemi di Sicurezza per l'Autenticazione Utente	26
6.7	Applicazione Pratica: Un Sistema Biometrico dell'Iride	26
6.8	Caso di Studio: Problemi di Sicurezza per i Sistemi ATM	27
7	Controllo degli Accessi	28
7.1	Principi del Controllo degli Accessi	28
7.2	Soggetti, Oggetti e Diritti di Accesso	29
7.3	Controllo degli Accessi Discrezionale (DAC)	29
7.4	Esempio Pratico: File Access Control in UNIX	30
7.5	Controllo degli Accessi Basato sui Ruoli (RBAC)	31
7.6	Controllo degli Accessi Basato sugli Attributi (ABAC)	32
7.7	Gestione delle Identità, Credenziali e Accessi (ICAM)	33
7.8	Framework di Fiducia (Trust Frameworks)	33
7.9	Caso di Studio: Sistema RBAC per una Banca	33
8	Programmi e Programmazione: Dalla Teoria alla Vulnerabilità	35
8.1	Sviste di Programmazione Non Intenzionali (Nonmalicious Oversight)	36
8.2	Codice Malevolo: Il Malware (Software Intenzionalmente Dannoso)	39
8.3	Contromisure per Sviluppatori: I Pilastri dell'Ingegneria del Software (Software Engineering)	41
8.4	Conclusioni: La Fallacia della "Security by Obscurity"	43

Concetti di Sicurezza Informatica

Definizione di Sicurezza Informatica (NIST)

La protezione garantita a un sistema informativo automatizzato al fine di raggiungere gli obiettivi applicabili di preservare l'integrità, la disponibilità e la riservatezza delle risorse del sistema informativo (inclusi hardware, software, firmware, informazioni/dati e telecomunicazioni).

Questa definizione introduce i tre obiettivi chiave, noti come **Triade CIA**:

- **Confidenzialità (Confidentiality):** preserva le restrizioni autorizzate all'accesso delle informazioni. Copre due concetti:
 - *Confidenzialità dei dati:* Assicura che le informazioni private non siano rese disponibili a individui non autorizzati.
 - *Privacy:* Assicura che gli individui controllino quali informazioni personali vengano raccolte, memorizzate e divulgate.
- **Integrità (Integrity):**
 - *Integrità dei dati:* Assicura che le informazioni e i programmi siano modificati solo in modo specificato e autorizzato.
 - *Integrità del sistema:* Assicura che un sistema svolga le sue funzioni previste in modo inalterato, libero da manipolazioni non autorizzate.
- **Disponibilità (Availability):** Assicura che i sistemi funzionino tempestivamente e che il servizio non venga negato agli utenti autorizzati.

Oltre alla Triade CIA, spesso si aggiungono due ulteriori concetti:

- **Autenticità:** La proprietà di essere genuini, verificabili e attendibili. Significa verificare l'identità degli utenti e la fonte degli input.
- **Responsabilità (Accountability):** La necessità che le azioni di un'entità possano essere ricondotte unicamente ad essa (supporta il non ripudio, l'analisi forense e la deterrenza).

1.1 Livelli di Impatto (FIPS 199) ed Esempi

In caso di violazione della sicurezza, i livelli di impatto si dividono in:

- **Basso:** Effetto avverso limitato (degrado funzionale lieve, danni minori, lievi perdite finanziarie).
- **Moderato:** Effetto avverso grave (significativo degrado delle funzioni primarie, danni o perdite finanziarie rilevanti).
- **Alto:** Effetto catastrofico (impossibilità di eseguire funzioni primarie, danni ingenti, o rischi per la vita umana).

Esempi pratici:

- *Confidenzialità:* Voti degli studenti (Alto), Informazioni di iscrizione (Moderato), Elenchi telefonici o directory pubbliche (Basso).
- *Integrità:* Dati sulle allergie dei pazienti ospedalieri (Alto), Forum pubblico su sito web (Moderato), Sondaggio online anonimo (Basso).
- *Disponibilità:* Servizi di autenticazione per sistemi critici (Alto), Sito web pubblico di un'università (Moderato), Directory telefonica online (Basso).

1.1.1 Le Sfide della Sicurezza Informatica

La sicurezza informatica è complessa per diversi motivi:

1. I requisiti sembrano semplici, ma i meccanismi per soddisfarli sono complessi.
2. Bisogna sempre considerare potenziali attacchi e ragionare in modi inaspettati.
3. Le procedure di sicurezza sono spesso controintuitive e complesse.
4. È difficile decidere *dove* posizionare le difese (fisicamente e logicamente).
5. Richiede la gestione sicura di informazioni segrete (es. chiavi di crittografia).
6. È una battaglia d'ingegno: l'attaccante deve trovare una sola debolezza, il difensore deve eliminarle tutte.
7. Utenti e manager non percepiscono i benefici finché non si verifica un guasto.
8. Richiede un monitoraggio costante.
9. Spesso viene aggiunta a posteriori, anziché essere integrata nel design iniziale.
10. Spesso è vista come un ostacolo all'efficienza e all'usabilità del sistema.

1.2 Un Modello per la Sicurezza Informatica

Terminologia Base (RFC 4949)

- **Avversario (agente di minaccia):** Un'entità che attacca o rappresenta una minaccia per un sistema.
- **Asset (Risorsa):** Hardware, software, dati, reti.
- **Vulnerabilità:** Un difetto o debolezza sfruttabile. Possono essere corrotti (integrità), divulgati (confidenzialità), o resi non disponibili (disponibilità).
- **Minaccia (Threat):** Potenziale violazione della sicurezza.
- **Attacco:** Un'azione derivante da una minaccia intelligente per evadere la sicurezza.
- **Contromisura:** Azione, dispositivo o procedura per ridurre minacce o vulnerabilità.
- **Rischio:** Probabilità che una minaccia sfrutti una vulnerabilità causando un danno.
- **Politica di sicurezza:** Un insieme di regole e pratiche che specificano o regolano il modo in cui un sistema o un'organizzazione fornisce servizi di sicurezza a proteggere le risorse di sistema sensibili e critiche.

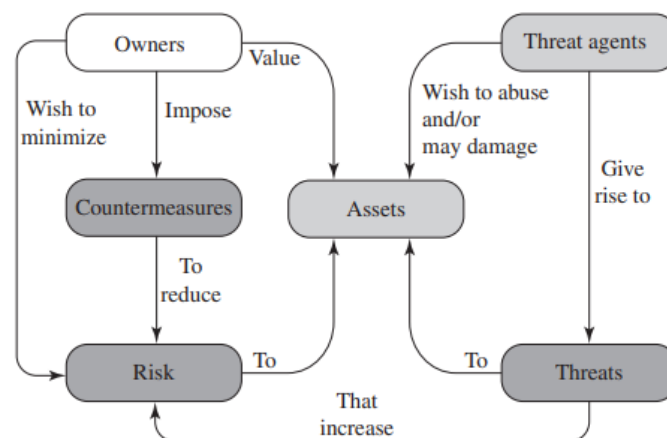


Figura 1: Concetti chiave di sicurezza informatica e loro relazioni (RFC 4949)

Gli **attacchi** possono essere:

- *Attivi:* alterano le risorse del sistema.
- *Passivi:* tentano di apprendere informazioni senza alterare il sistema.
- *Interni/Esterni:* avviati da chi ha già autorizzazioni vs avviati da soggetti non legittimati dall'esterno.

1.3 Minacce, Attacchi e Asset

1.3.1 Conseguenze delle Minacce e Azioni (Attacchi)

La RFC 4949 descrive quattro principali conseguenze:

1. Divulgazione Non Autorizzata (Confidenzialità):

- *Esposizione*: Rilascio diretto a entità non autorizzate (es. errore umano).
- *Intercettazione*: Accesso diretto ai dati in transito su una rete (sniffing).
- *Inferenza*: Dedurre informazioni sensibili analizzando il traffico o incrociando dati anonimi.
- *Intrusione*: Superare i controlli di accesso per ottenere dati.

2. Inganno / Deception (Integrità):

- *Mascheramento (Masquerade)*: Farsi passare per un'entità autorizzata.
- *Falsificazione*: Modificare o inserire dati falsi.
- *Ripudio*: Negare di aver inviato/ricevuto dati o azioni compiute.

3. Interruzione / Disruption (Disponibilità o Integrità del sistema):

- *Incapacitazione*: Distruzione fisica o disabilitazione tramite malware.
- *Corruzione*: Modifica del funzionamento del sistema (es. inserimento di backdoor).
- *Ostruzione*: Sovraccarico del sistema o taglio dei collegamenti (es. Denial of Service).

4. Usurpazione (Integrità del sistema):

- *Appropriazione indebita*: Furto di servizi (es. usare macchine infette per botnet).
- *Uso improprio*: Eludere o disabilitare le funzioni di sicurezza.

1.3.2 Minacce per Categoria di Asset

- **Hardware**: Il rischio principale è la perdita di *disponibilità* (furto o danni fisici). Richiede sicurezza fisica.
- **Software**: Anche qui minacce alla *disponibilità* (cancellazione) e all'*integrità* (modifiche malevole come virus). Inoltre, furto (pirateria).
- **Dati**: Richiede disponibilità (backup), confidenzialità (crittografia) e integrità. Un problema noto sono i database statistici, dove aggregazioni ripetute possono rivelare dati personali (inferenza).
- **Linee di Comunicazione e Reti**:
 - *Attacchi Passivi*: Rilascio di contenuti e Analisi del traffico (difficili da rilevare, ma si possono prevenire con la crittografia).

- *Attacchi Attivi*: Replay, Mascheramento, Modifica dei messaggi e Denial of Service (difficili da prevenire totalmente; l'obiettivo primario è rilevarli e mitigarli).

1.4 Requisiti Funzionali di Sicurezza

Il FIPS 200 (Minimum Security Requirements for Federal Information and Information Systems) elenca 17 aree essenziali per proteggere la Triade CIA. Queste aree si dividono in:

- **Misure Tecniche:** Controllo accessi, Identificazione e autenticazione, Protezione del sistema e delle comunicazioni, Integrità del sistema e delle informazioni.
- **Misure Manageriali:** Formazione e consapevolezza, Valutazione dei rischi, Sicurezza del personale, Pianificazione di contingenza, Protezione fisica/ambientale.
- **Aree Sovrapposte:** Gestione della configurazione, Risposta agli incidenti, Protezione dei media.

Come notano gli esperti: “Se pensi che la tecnologia da sola possa risolvere i tuoi problemi di sicurezza, allora non capisci i problemi e non capisci la tecnologia”. Serve un mix manageriale e tecnico.

1.5 Principi Fondamentali di Progettazione della Sicurezza

Non esiste un metodo infallibile, quindi ci si affida a 13 principi guida ampiamente accettati:

1. **Economia del Meccanismo (Economy of mechanism):** Il design deve essere semplice e ridotto al minimo. Più è semplice, meno vulnerabilità ci saranno e più facile sarà testarlo.
2. **Default Sicuro (Fail-safe defaults):** Le decisioni di accesso dovrebbero basarsi su permessi espliciti anziché su esclusioni. Il default deve essere l'accesso negato.
3. **Mediazione Completa (Complete mediation):** Ogni singolo accesso deve essere controllato dal sistema di sicurezza, senza fare affidamento ciecamente sulla cache.
4. **Progettazione Aperta (Open design):** Gli algoritmi (come quelli di crittografia) non devono essere tenuti segreti, ma aperti al controllo pubblico. Solo le chiavi devono essere segrete.
5. **Separazione dei Privilegi (Separation of privilege):** Richiedere molteplici attributi per concedere privilegi (es. autenticazione a più fattori) o dividere un programma in parti limitate ai soli privilegi necessari.
6. **Principio del Minimo Privilegio (Least privilege):** Ogni utente/processo deve operare solo con i privilegi strettamente necessari per svolgere i propri compiti temporanei (es. Role-Based Access Control).

7. **Minimo Meccanismo Comune (Least common mechanism):** Minimizzare le funzioni condivise da utenti diversi per ridurre percorsi di comunicazione non voluti.
8. **Accettabilità Psicologica (Psychological acceptability):** La sicurezza non deve interferire eccessivamente con il lavoro dell'utente; deve essere trasparente e usabile, altrimenti l'utente cercherà di aggirarla.
9. **Isolamento (Isolation):** Separare sistemi pubblici da dati critici (fisicamente/logicamente), isolare i processi dei vari utenti e nascondere i meccanismi di sicurezza al pubblico.
10. **Incapsulamento (Encapsulation):** Deriva dalla programmazione a oggetti; nascondere la struttura interna e consentire l'accesso solo tramite entry point protetti e designati.
11. **Modularità (Modularity):** Sviluppare funzioni di sicurezza come moduli separati (es. moduli crittografici) e rendere l'architettura aggiornabile a pezzi senza ridisegnare tutto.
12. **Stratificazione (Layering / Defense in Depth):** Usare molteplici approcci di protezione sovrapposti. Se una difesa fallisce, un'altra protegge il sistema.
13. **Minimo Stupore (Least astonishment):** Il sistema o l'interfaccia utente dovrebbe rispondere sempre nel modo in cui l'utente si aspetta intuitivamente, riducendo gli errori umani.

1.6 Superfici di Attacco e Alberi di Attacco

Per valutare le minacce si utilizzano due concetti chiave.

1.6.1 Superfici di Attacco

Consiste nelle vulnerabilità raggiungibili e sfruttabili di un sistema (es. porte aperte, codice in ascolto, interfacce web, dipendenti vulnerabili). Le categorie sono:

- **Superficie di Rete:** Vulnerabilità nei protocolli esposti su LAN/WAN.
- **Superficie Software:** Vulnerabilità nelle applicazioni, nel SO o nei server Web.
- **Superficie Umana:** Vulnerabilità date da dipendenti (errori umani, social engineering).

Ridurre la superficie d'attacco e stratificare la difesa (Defense in Depth) abbassa significativamente il rischio.

1.6.2 Alberi di Attacco (Attack Trees)

Un albero di attacco è una struttura gerarchica ad albero che rappresenta le tecniche per sfruttare le vulnerabilità:

- Il **nodo radice** è l'obiettivo finale dell'attaccante.
- I **nodi foglia** sono gli eventi iniziali o le azioni base per avviare l'attacco.

- Si usano nodi di tipo AND (tutti i sotto-obiettivi devono riuscire) o OR (basta che un sotto-obiettivo riesca).

Prendendo come esempio il furto di un account bancario online, le strategie tipiche (nodi OR) includono: Compromissione delle credenziali dell'utente (social engineering, trojan), Iniezione di comandi nel traffico (man-in-the-middle), Indovino delle credenziali (brute force), Violazione delle policy di sicurezza interne (insider), e Dirottamento di sessione (session hijacking).

1.7 Strategia di Sicurezza Informatica

Una strategia completa si divide in tre aspetti principali:

1. **Policy di Sicurezza (Cosa fare?):** Un insieme formale di regole e pratiche. Per definirla bisogna valutare il valore degli asset, le vulnerabilità e i trade-off continui tra *Facilità d'uso vs. Sicurezza* e *Costo della sicurezza vs. Costo del fallimento e ripristino*. È fondamentalmente una decisione di business.
2. **Implementazione e Meccanismi (Come farlo?):**
 - *Prevenzione*: Prevenire l'attacco alla radice (es. crittografia forte).
 - *Rilevamento (Detection)*: Riconoscere un attacco in corso (es. Intrusion Detection Systems).
 - *Risposta*: Bloccare l'attacco e mitigare i danni in tempo reale.
 - *Ripristino (Recovery)*: Ristabilire l'integrità dei dati o del sistema (es. Backup).
3. **Garanzia e Valutazione (Assurance and Evaluation) (Funziona davvero?):**
 - *Garanzia (Assurance)*: Il livello di confidenza che i meccanismi (tecnici e operativi) funzionino come previsto. Non è una prova matematica assoluta, ma un alto grado di sicurezza.
 - *Valutazione (Evaluation)*: Il processo pratico di test ed esame sistematico dei sistemi rispetto a determinati criteri di sicurezza.

Crittografia Simmetrica e Confidenzialità

2.1 Principi Fondamentali e Attacchi

Crittografia Simmetrica

Nota anche come crittografia convenzionale o a chiave singola, è stato l'unico tipo di crittografia in uso prima degli anni '70. Utilizza una sola chiave condivisa sia per cifrare che per decifrare i dati. È il metodo più diffuso per garantire la confidenzialità.

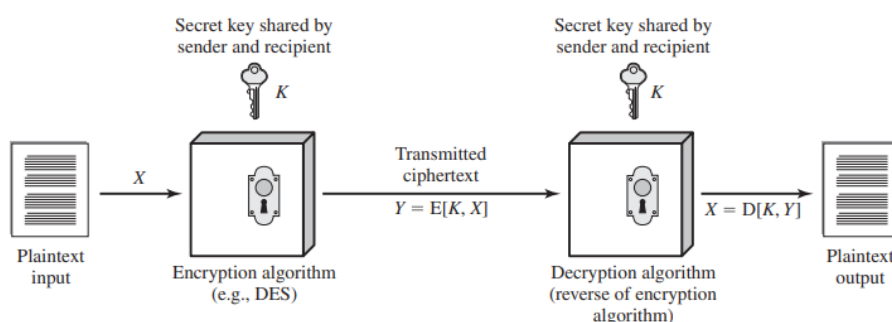


Figura 2: Schema semplificato di crittografia simmetrica

Gli elementi fondamentali di uno schema simmetrico sono cinque:

- **Testo in chiaro (Plaintext):** Il messaggio originale o i dati forniti in input all'algoritmo.
- **Algoritmo di crittografia:** Esegue varie sostituzioni e trasformazioni sul testo in chiaro.
- **Chiave segreta:** L'input dell'algoritmo; guida l'esatta natura delle trasformazioni indipendentemente dal testo in chiaro.
- **Testo cifrato (Ciphertext):** Il messaggio incomprensibile prodotto in output. Dipende sia dal testo in chiaro che dalla chiave.
- **Algoritmo di decrittografia:** L'algoritmo di crittografia eseguito al contrario, che prende in input il testo cifrato e la *stessa* chiave segreta per riprodurre il testo originale.

I sistemi crittografici sono classificati lungo tre dimensioni indipendenti:

1. **Tipo di operazioni:** *Sostituzione* (ogni elemento del plaintext viene mappato in un altro elemento) e *Trasposizione* (gli elementi vengono riarrangiati). Le operazioni devono essere reversibili. I sistemi moderni ("sistemi prodotto") usano molteplici stadi di entrambe.
2. **Numero di chiavi:** Simmetrica (chiave singola) o Asimmetrica (a chiave pubblica).
3. **Elaborazione del testo:** A blocchi (elabora un blocco di elementi alla volta) o a flusso (elabora gli elementi in modo continuo, uno alla volta).

Per un uso sicuro servono un **algoritmo forte** e un modo sicuro per **scambiarsi e proteggere la chiave**. Un algoritmo si definisce **computazionalmente sicuro** se il costo per decifrarlo supera il valore delle informazioni, oppure il tempo richiesto per forzarlo supera il ciclo di vita utile delle informazioni stesse.

Gli attacchi principali per forzare un cifrario sono la forza bruta (provare tutte le chiavi possibili) e la **crittanalisi** (sfruttare la natura dell'algoritmo). Gli attacchi si dividono in base alle informazioni possedute dall'analista:

- **Solo testo cifrato (Ciphertext only):** Si conosce solo l'algoritmo e il testo cifrato. Richiede analisi statistica ed è il più difficile.
- **Testo in chiaro noto (Known plaintext):** Si possiedono coppie di testo in chiaro e relativo testo cifrato generate con la stessa chiave segreta.
- **Testo in chiaro scelto (Chosen plaintext):** L'analista può far cifrare al sistema un messaggio da lui scelto.
- **Testo cifrato scelto (Chosen ciphertext):** L'analista fornisce un testo cifrato fittizio e si fa restituire il decifrato.
- **Testo scelto (Chosen text):** L'analista manipola a piacimento sia input in chiaro che in cifrato.

2.2 Cifrari a Blocchi e Algoritmi Principali

Un cifrario a blocchi elabora l'input in chiaro suddividendolo in blocchi di dimensione fissa, producendo per ciascuno un blocco di testo cifrato di pari dimensione. Molti algoritmi a blocchi usano la **Struttura di Feistel** (1973): il blocco in chiaro (lungo $2w$ bit) viene diviso in due metà (L_0 e R_0) che passano attraverso n round. In ogni round i , la metà destra (R_{i-1}) passa a una funzione F parametrizzata da una sotto-chiave K_i . L'output di F è unito in XOR ($\oplus$) con la metà sinistra (L_{i-1}), e le due metà vengono scambiate.

La progettazione dipende dalla dimensione del blocco/chiave (maggiore dimensione = maggiore sicurezza ma minore velocità), numero di round (tipicamente 16), e complessità dell'algoritmo di generazione delle sotto-chiavi.

I principali algoritmi simmetrici a blocchi sono:

- **DES (Data Encryption Standard):** Adottato nel 1977, usa blocchi da 64 bit, una chiave da 56 bit e 16 round di Feistel. Oggi è inadeguato: con le potenze di calcolo attuali, un attacco a forza bruta provando tutte le 2^{56} combinazioni può forzare il DES in poche ore o giorni.
- **Triple DES (3DES):** Progettato per salvare il DES, ripete l'algoritmo tre volte in sequenza *Cifra-Decifra-Cifra (EDE)* usando due o tre chiavi uniche (112 o 168 bit effettivi):

$$C = E(K_3, D(K_2, E(K_1, P)))$$

Usare la decrittografia nel passaggio centrale garantisce la *retrocompatibilità* col vecchio DES ponendo $K_1 = K_2 = K_3$. Supera la vulnerabilità alla forza bruta mantenendo un algoritmo super-testato, ma è molto lento via software e mantiene un blocco piccolo (64 bit).

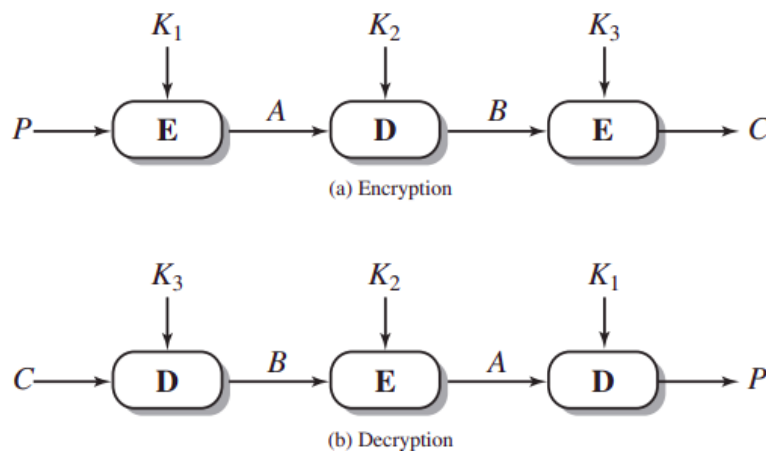


Figura 3: Schema di Triple DES (3DES)

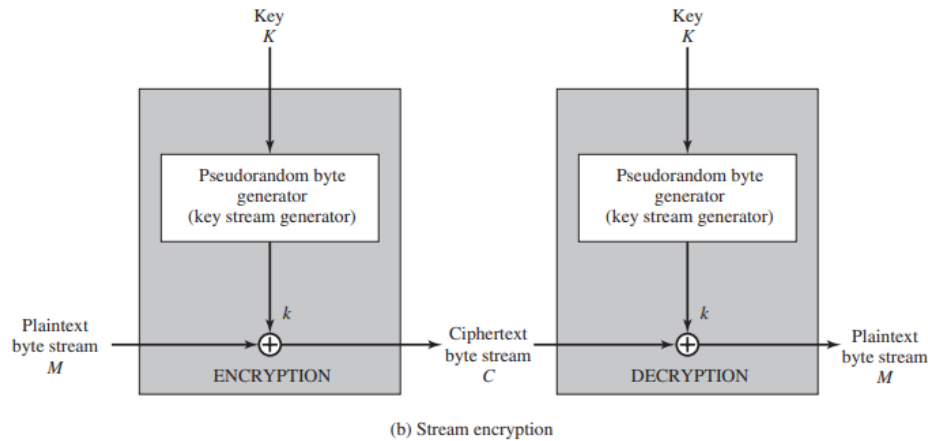
- **AES (Advanced Encryption Standard):** Selezionato dal NIST nel 2001 (basato sull'algoritmo Rijndael) per rimpiazzare il 3DES. Utilizza blocchi ampi da 128 bit e chiavi da 128, 192 o 256 bit. **Non** è una rete di Feistel: elabora l'intero blocco in parallelo tramite una matrice 4x4 detta *State*. Attraversa 10 round (per chiave 128 bit) composti da 4 fasi reversibili:

1. **Substitute Bytes (SubBytes):** Sostituzione byte per byte tramite S-box.
2. **Shift Rows:** Permutazione circolare a livello di riga.
3. **Mix Columns:** Altera ogni byte di una colonna in funzione di tutti i byte della colonna.
4. **Add Round Key:** XOR tra lo State e la porzione di chiave espansa.

2.3 Modalità di Operazione per Cifrari a Blocchi

Nella realtà, i dati (come email o database) sono più grandi di un singolo blocco. Il NIST definisce diverse *Modes of Operation*:

- **Electronic Codebook (ECB):** Ogni blocco è cifrato in modo indipendente. *Il problema:* Non è sicuro per messaggi lunghi o strutturati. Se il testo originale contiene pattern ricorrenti, blocchi in chiaro identici produrranno blocchi cifrati identici. È come avere un enorme vocabolario visibile all'analista.



- **Cipher Block Chaining (CBC):** L'input dell'algoritmo è lo XOR tra il blocco in chiaro attuale e il ciphertext precedente ($C_j = E(K, (C_{j-1} \oplus P_j))$). Nasconde i pattern ripetuti. Per il primo blocco serve un Vettore di Inizializzazione (IV) che deve essere protetto dalle manipolazioni.
- **Cipher Feedback (CFB):** Trasforma il cifrario a blocchi in un cifrario a flusso. Ideale per trasmissioni in tempo reale.
- **Counter (CTR):** Cifra un *contatore* e ne fa lo XOR col testo in chiaro. Offre un'efficienza enorme: permette esecuzione in **parallelo**, garantisce *Random Access* (si può decifrare il blocco *i*-esimo saltando i precedenti) e richiede di implementare solo la funzione di cifratura.

2.4 Cifrari a Flusso e RC4

A differenza dei cifrari a blocchi, un cifrario a flusso elabora gli elementi di input in modo continuo. Una chiave alimenta un **generatore di bit pseudocasuali** che produce un **keystream** (flusso di chiavi), unito al plaintext tramite operazione logica **XOR bit a bit**. Sono idealmente più veloci e richiedono meno codice, perfetti per le telecomunicazioni in tempo reale. *Vulnerabilità nota:* La chiave di un cifrario a flusso non deve mai essere riutilizzata per testi diversi, altrimenti lo XOR dei ciphertext espone lo XOR dei testi originali.

L'Algoritmo RC4:

Progettato da Ron Rivest, è velocissimo in software e usato in SSL/TLS e WEP. Inizializza un vettore di stato *S* di 256 byte basato su una chiave variabile, effettuando una complessa permutazione. Genera poi il flusso scorrendo *S*, scambiando costantemente le celle e estraendo un byte pseudo-casuale da usare nello XOR col testo. Il problema del protocollo WEP non era l'RC4 in sé, ma la generazione debole delle chiavi.

2.5 Posizionamento della Sicurezza e Distribuzione delle Chiavi

I dati possono essere cifrati in due punti della rete:

- **Link Encryption (Cifratura di Linea):** Tutto il traffico è cifrato tra i nodi. I pacchetti vengono decifrati in chiaro all'interno di ogni switch/router per leggere l'indirizzo, esponendo i nodi intermedi.
- **End-to-End Encryption:** Eseguita dagli host finali. I dati (payload) restano cifrati per tutto il tragitto. Tuttavia, l'header viaggia in chiaro affinché i router possano smistarlo, permettendo l'*analisi del traffico* agli avversari.

Key Distribution Center (KDC) e Automazione

Per comunicare, A e B devono scambiarsi una chiave. Per evitare lo scambio manuale, si usa un KDC fiduciario. Ogni host condivide con il KDC una **Permanent Key**.

1. L'Host A invia una richiesta per B al KDC.
2. Il KDC approva e genera una **Session Key** temporanea.
3. Il KDC cifra la Session Key con la Permanent Key di A e la invia ad A; parallelamente la cifra con la Permanent Key di B e la invia a B.
4. Ora A e B condividono un segreto e possono trasmettere dati cifrati, distruggendo la chiave a fine sessione.

3

Autenticazione dei Messaggi e Funzioni Hash

3.1 Principi di Autenticazione e MAC

L'autenticazione del messaggio protegge dagli attacchi attivi (falsificazione di dati). Permette di verificare che un messaggio: non sia stato alterato (Integrità), provenga dalla fonte dichiarata (Autenticità) e sia nella sequenza corretta. La crittografia simmetrica non basta da sola (es. i blocchi possono essere riordinati in ECB).

Spesso si preferisce inviare il messaggio in chiaro allegando solo un **tag di autenticazione** perché: è più economico nei broadcast, riduce il carico sui nodi che non hanno tempo di decifrare tutto, e permette di verificare l'eseguibilità di un programma senza sprechi di CPU.

Message Authentication Code (MAC):

Un MAC usa una chiave segreta condivisa per generare un piccolo blocco di dati a dimensione fissa, $MAC = F(K_{AB}, M)$, appeso al messaggio. Il destinatario ricalcola e confronta il MAC. Questo garantisce integrità (l'attaccante non ha la chiave per ricalcolarlo) e provenienza sicura.

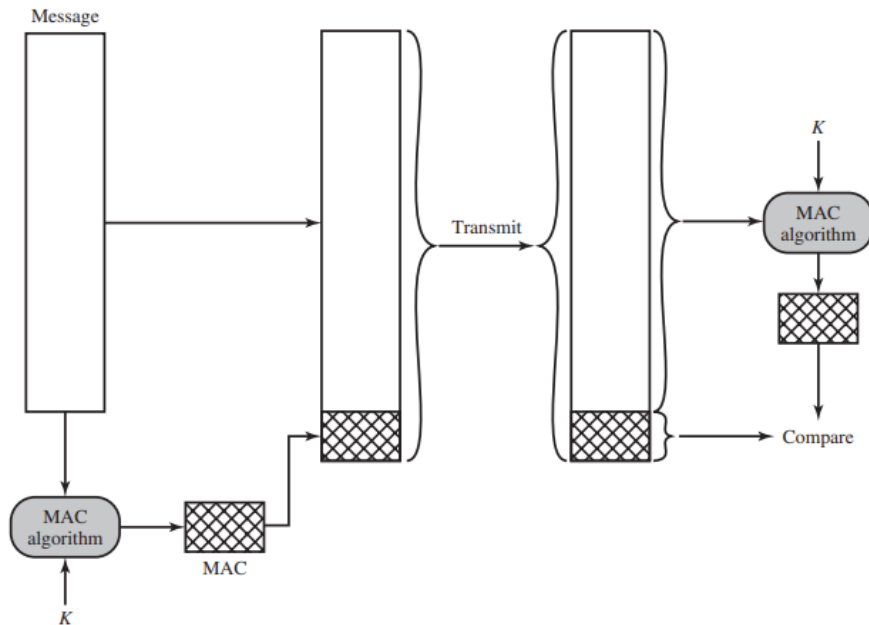


Figura 4: Schema di autenticazione con MAC

HMAC (Hash-based MAC):

Poiché le funzioni hash sono più veloci dei cifrari, RFC 2104 ha definito HMAC, che usa l'hash come "scatola nera". Evita del tutto i costi di calcolo crittografico pesanti. L'HMAC è definito come:

$$HMAC(K, M) = H((K^+ \oplus opad) \parallel H((K^+ \oplus ipad) \parallel M))$$

Sfrutta operazioni di padding della chiave (K^+), *ipad* e *opad* per eseguire doppi hashing concatenati. La sua genialità è che una forgiatura richiede di forzare direttamente l'hash trovando collisioni, ma siccome la chiave K è ignota all'attaccante, anche hash datati come MD5 offrono ancora sicurezza adeguata e grande velocità in ambito HMAC.

3.2 Funzioni Hash Sicure

Funzione Hash

Accetta un messaggio M di lunghezza variabile e produce un'impronta digitale (message digest) $H(M)$ di lunghezza fissa. Tipicamente, il messaggio viene "riempito" (padding) e include un campo con la lunghezza originale. A differenza del MAC, l'algoritmo in sé non utilizza chiavi segrete.

I modi per usare l'hash nell'autenticazione sono: cifratura simmetrica dell'hash, firma digitale (cifratura dell'hash con chiave privata) o HMAC (concatenazione hash e chiave).

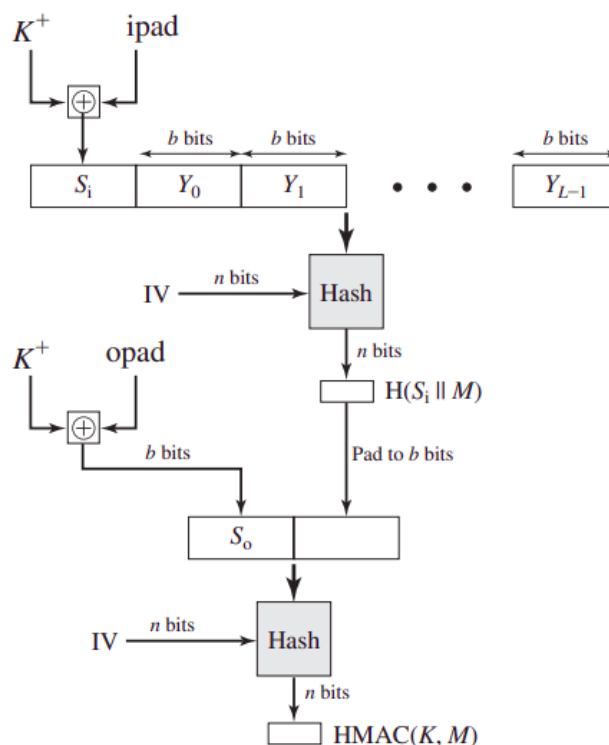


Figura 5: Schema di HMAC

Requisiti delle Funzioni Hash:

Una funzione H valida deve avere 6 proprietà. Le prime 3 pratiche: (1) Applicabile a blocchi di ogni dimensione, (2) Produce output di lunghezza fissa, (3) È facile e veloce da calcolare in HW/SW. Le ultime 3 sono i pilastri di sicurezza:

- **Resistenza alla preimmagine (One-way):** Dato l'hash h , è computazionalmente infattibile trovare il messaggio originale x . *Utilità:* Impedisce di estrarre chiavi segrete concatenate.
- **Seconda resistenza alla preimmagine (Debole resistenza alle collisioni):** Dato x , è infattibile trovare un diverso y con lo stesso hash. *Utilità:* Impedisce la contraffazione di messaggi in transito.
- **Resistenza alle collisioni (Forte resistenza alle collisioni):** È infattibile trovare una *qualsiasi* coppia (x, y) che produca lo stesso hash. *Utilità:* Protegge dalle truffe in cui l'attaccante genera due messaggi con stesso hash facendone firmare uno innocuo alla vittima.

Funzioni Hash Semplici e RXOR:

La forma più banale è lo XOR bit a bit di ogni blocco ($C_i = b_{i1} \oplus \dots \oplus b_{im}$). Tuttavia, i dati testuali hanno schemi prevedibili (es. i bit superiori dell'ASCII a zero), azzerando intere porzioni dell'hash. Migliorando con lo *Shift Circolare (RXOR)* si risolvono i pattern casuali, ma la crittanalisi dimostra che rimane totalmente inutile contro attacchi intenzionali (es. manipolazioni di testo in chiaro o permutazioni in CBC mode non cambiano l'hash finale).

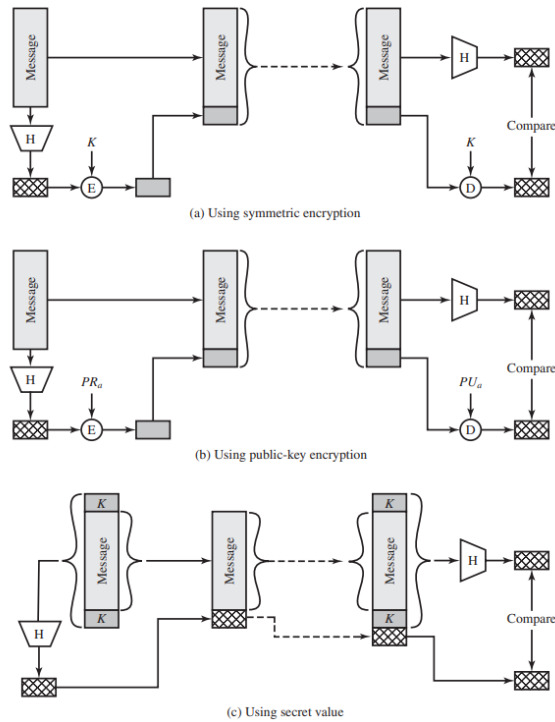


Figura 6: Schema di autenticazione con funzioni hash

	Bit 1	Bit 2	• • •	Bit n
Block 1	b_{11}	b_{21}		b_{n1}
Block 2	b_{12}	b_{22}		b_{n2}
	•	•	•	•
	•	•	•	•
	•	•	•	•
Block m	b_{1m}	b_{2m}		b_{nm}
Hash code	C_1	C_2		C_n

L'algoritmo SHA (Secure Hash Algorithm):

Dato il superamento storico di MD5 e SHA-1 (digest 160 bit), oggi si usano le famiglie **SHA-2** (SHA-256, 384, 512) o il sostituto "drop-in" **SHA-3**.

Confronto Parametri Famiglia SHA

- **SHA-1:** Digest 160 bit | Blocco 512 bit | Parola 32 bit | 80 passaggi (Sicurezza: 80 bit)
- **SHA-256:** Digest 256 bit | Blocco 512 bit | Parola 32 bit | 64 passaggi (Sicurezza: 128 bit)
- **SHA-384:** Digest 384 bit | Blocco 1024 bit | Parola 64 bit | 80 passaggi (Sicurezza: 192 bit)
- **SHA-512:** Digest 512 bit | Blocco 1024 bit | Parola 64 bit | 80 passaggi (Sicurezza: 256 bit)

Dettaglio di **SHA-512**: Il messaggio viene riempito (padding col bit '1' e poi zeri) e viene appesa la sua lunghezza originale (128 bit). L'elaborazione sfrutta un buffer diviso in registri inizializzati con costanti derivate dalle radici quadrate di numeri primi.

L'algoritmo elabora blocchi da 1024 bit su **80 round**, usando shift, operatori logici e costanti additive basate su radici cubiche dei numeri primi. Questo produce un "effetto valanga" tale che risalire a una preimmagine richiede 2^{512} operazioni.

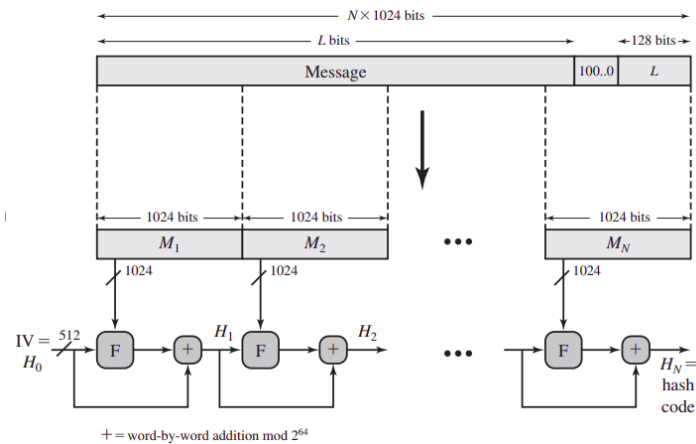


Figura 7: Message digest generation in SHA-512

L'hash trova largo uso anche nel mascheramento delle **Password** nei sistemi operativi, e negli **Intrusion Detection Systems** (dove il ricalcolo degli hash dei file di sistema permette di scoprire manipolazioni esterne).

4

Crittografia a Chiave Pubblica e Firme Digitali

4.1 Principi e Requisiti dell'Asimmetrica

Proposta da Diffie e Hellman nel 1976, la crittografia asimmetrica utilizza due chiavi separate basate su funzioni matematiche anziché su manipolazioni bit-level. **Falsi miti:** Non è più sicura della simmetrica (dipende solo dai bit della chiave), non la rende obsoleta (ha un carico computazionale enorme, quindi la simmetrica resta essenziale) e non semplifica la distribuzione delle chiavi (serve comunque un'infrastruttura fidata).

Lo schema prevede 6 ingredienti: Plaintext, Algoritmo crittografico, Chiave Pubblica (nota a tutti), Chiave Privata (segreta), Ciphertext, e Algoritmo di decrittografia. A seconda della chiave usata per cifrare, fornisce:

- **Confidenzialità:** Alice cifra con la *chiave pubblica* di Bob. Solo Bob con la sua *chiave privata* può leggere.
- **Autenticazione/Firma:** Bob cifra con la *sua chiave privata*. Tutti, usando la pubblica di Bob, possono decifrare e confermare l'origine inequivocabile.

I requisiti matematici per generare le coppie impongono che sia facile generarle, facile cifrare/decifrare per i legittimi proprietari, ma **computazionalmente infattibile** dedurre la privata dalla pubblica o risalire al testo in chiaro dal solo testo cifrato.

4.2 L'algoritmo RSA

Sviluppato da Rivest, Shamir e Adleman al MIT nel 1977, cifra interi M minori di n tramite esponenziazione modulare:

$$C = M^e \pmod{n}$$

$$M = C^d \pmod{n} = (M^e)^d \pmod{n} = M^{ed} \pmod{n}$$

Chiave Pubblica $PU = \{e, n\}$, Chiave Privata $PR = \{d, n\}$. La validità poggia sulla **funzione totiente di Eulero** $\phi(n)$. **Generazione chiavi:**

1. Selezionare due enormi numeri primi p e q .
2. Calcolare il modulo $n = p \times q$ e $\phi(n) = (p-1)(q-1)$.
3. Scegliere e coprimo a $\phi(n)$.
4. Calcolare d come l'inverso moltiplicativo di e modulo $\phi(n)$.

Esempio: $p = 17, q = 11 \rightarrow n = 187, \phi(n) = 160$. Scelgo $e = 7$. Allora $d = 23$. Cifrando $M = 88$, il calcolo $88^7 \pmod{187}$ viene eseguito scomponendo e sfruttando l'aritmetica modulare in cascata, restituendo $C = 11$.

La **sicurezza dell'RSA** sta nella difficoltà estrema di fattorizzare l'enorme n per ritrovare p e q . Sebbene algoritmi come il Generalized Number Field Sieve (GNFS) abbiano forzato sfide fino a 663 bit, le attuali chiavi a 1024 o 2048 bit sono blindate. L'RSA deve guardarsi anche dai **Timing Attacks** (osservazione del tempo di esecuzione hardware). Si risolvono con la tecnica del **Blinding**: il messaggio viene "accecato" con un numero casuale segreto r prima di essere processato, azzerando le informazioni temporali estraibili, per poi essere ripristinato a fine calcolo ($M = M' \times r^{-1} \pmod{n}$).

4.3 Scambio di Chiavi Diffie-Hellman

Questo protocollo serve **esclusivamente** per permettere a due utenti di concordare un segreto condiviso (una chiave simmetrica) su un canale insicuro, e si basa sulla durezza del calcolo dei **logaritmi discreti** (es. trovare l'esponente i in $b = a^i \pmod{p}$). Dati un numero primo grande q e una sua radice primitiva a :

1. Alice sceglie $X_A < q$ (privato) e invia a Bob $Y_A = a^{X_A} \pmod{q}$.
2. Bob sceglie $X_B < q$ (privato) e invia ad Alice $Y_B = a^{X_B} \pmod{q}$.
3. Alice calcola $K = (Y_B)^{X_A} \pmod{q}$. Bob calcola $K = (Y_A)^{X_B} \pmod{q}$.

4. Entrambi ottengono la stessa magica K . L'attaccante che osserva Y_A e Y_B dovrebbe calcolare un infattibile logaritmo discreto per violarli.

L'Attacco Man-in-the-Middle (MITM)

Il tallone d'Achille di Diffie-Hellman puro è che **non autentica i partecipanti**. Darth può interpersi al centro: riceve le chiavi da Alice e Bob, genera proprie chiavi finte e stabilisce due connessioni segrete separate. Alice e Bob comunicano pensando di essere sicuri, ma Darth al centro decifra, legge e ricifra tutto collaudando chiavi parallele. Serve abbinare il protocollo a firme digitali per risolverlo.

4.4 Firme Digitali e Certificati

Per garantire il non-ripudio, si usa la **Firma Digitale**: si crea l'hash del documento e si cifra *solo l'hash* con la propria chiave privata. Il destinatario ricalcola l'hash e lo confronta decifrando la firma con la chiave pubblica nota. Nota bene: la firma non oscura il messaggio, che viaggia in chiaro.

Come essere certi di possedere la vera chiave pubblica di Bob? Attraverso lo standard **X.509: i Certificati a Chiave Pubblica**. Una Certificate Authority (CA) fidata lega l'identità dell'utente alla sua chiave pubblica, e sigilla il pacchetto firmandolo con la propria inattaccabile chiave privata.

4.5 Altri Algoritmi Asimmetrici

- **DSS (Digital Signature Standard)**: FIPS PUB 186. Impiega l'algoritmo DSA ma, a differenza di RSA, è stato costruito per compiere **esclusivamente** firme digitali. Non sa scambiare chiavi o cifrare.
- **ECC (Elliptic-Curve Cryptography)**: Basato sulle curve ellittiche. È il grande sfidante, capace di offrire la stessa formidabile sicurezza di RSA ma usando chiavi immensamente più corte, diventando vitale per sgravare CPU di smartphone ed e-commerce. Paga solo lo scotto di una minore maturità storica.

Strumenti Aggiuntivi e Applicazioni

5.1 Busta Digitale (Digital Envelope)

Inviare enormi file interamente in RSA devasterebbe le prestazioni. Si usa un approccio combinato: Bob genera una *chiave simmetrica monouso* (velocissima) e ci cifra il messaggio pesante (es. AES). Dopodiché cifra solo quella piccola chiave monouso con l'*RSA (chiave pubblica di Alice)*. Invia ad Alice la "Busta" contenente i dati AES e la key cifrata. Alice estrarrà la chiave simmetrica con la sua privata e decifrerà i dati all'istante.

5.2 Numeri Casuali e Pseudocasuali

I numeri casuali sono il carburante per session keys, nonces e generazione di prime numbers. Servono *Casualità statistica* (distribuzione uniforme) e *Imprevedibilità*. I **PRNG** (Pseudorandom) usano calcoli matematici su un seme (seed): ottimi ma teoricamente deducibili. I **TRNG** (True Random) misurano vera entropia fisica caotica (decadimento radioattivo, rumore termico, LavaRnd, Intel DRNG).

5.3 Protezione dei Dati Archiviati (Data at Rest)

Proteggere il traffico di rete non salva dagli smarrimenti fisici dei dispositivi. Famosi disastri coinvolgono nastri e portatili governativi o ospedalieri non cifrati. Soluzioni impiegate:

1. **Software locali (es. PGP):** File specifici cifrati e decifrati in memoria tramite password.
2. **Appliance Back-end:** Hardware massiccio nascosto che cifra e decifra i flussi verso lo storage centrale senza pesare sui server, invisibilmente.
3. **Nastri cifrati in HW:** I backup offline viaggiano nei camion con il contenuto intrinsecamente illeggibile.
4. **Cifratura in background del Volume:** Sistemi come BitLocker presentano volumi virtuali: l'utente fa un solo login e il sistema cifra e decifra l'intero SSD "on-the-fly" per ogni operazione del sistema operativo.

Autenticazione dell'Utente

Nella maggior parte dei contesti di sicurezza informatica, l'autenticazione dell'utente rappresenta il blocco fondamentale e la principale linea di difesa. È la base per la maggior parte dei controlli di accesso e per la responsabilità (accountability) degli utenti.

Definizione di Autenticazione dell'Utente

Il processo di verifica di un'identità rivendicata da (o per) un'entità di sistema. Consiste in due fasi sequenziali:

- **Fase di Identificazione:** Presentazione di un identificatore al sistema di sicurezza (es. un nome utente o ID). L'identificazione è il mezzo attraverso il quale un utente fornisce un'identità dichiarata al sistema.
- **Fase di Verifica:** Presentazione o generazione di informazioni di autenticazione che corroborano il legame tra l'entità e l'identificatore (es. una password tenuta segreta). L'autenticazione è il mezzo per stabilire la validità di tale dichiarazione.

6.1 Principi dell'Autenticazione Elettronica dell'Utente

L'autenticazione elettronica è il processo volto a stabilire fiducia nelle identità presentate elettronicamente a un sistema informativo. I sistemi utilizzano l'identità autenticata per determinare se l'individuo è autorizzato a eseguire particolari funzioni, come transazioni su database o accesso a risorse di rete.

Modello per l'Autenticazione Elettronica dell'Utente

Il modello di riferimento per l'autenticazione elettronica definisce un'architettura generale che coinvolge diverse entità:

1. **Registration Authority (RA):** Un'entità fidata che stabilisce e garantisce l'identità di un richiedente, verificandone i requisiti.
2. **Credential Service Provider (CSP):** Rilascia una credenziale elettronica all'iscritto. La credenziale lega in modo autorevole un'identità a un *token* posseduto dall'utente.
3. **Claimant (Richiedente) e Verifier (Verificatore):** Il claimant è la parte che deve essere autenticata, mentre il verifier è il sistema che controlla e verifica l'identità.
4. **Relying Party (RP):** La parte (es. un'applicazione) che si affida alle affermazioni (assertions) del verificatore per prendere decisioni finali sul controllo degli accessi.

Mezzi di Autenticazione

Esistono quattro mezzi generali per autenticare l'identità di un utente:

1. **Qualcosa che l'individuo conosce:** Esempi includono password, PIN (Personal Identification Number) o risposte a domande di sicurezza predefinite.
2. **Qualcosa che l'individuo possiede (Token):** Oggetti fisici come chiavi elettroniche, smart card o token hardware.
3. **Qualcosa che l'individuo è (Biometria statica):** Caratteristiche fisiche univoche come impronte digitali, pattern della retina, iride o geometria del volto.
4. **Qualcosa che l'individuo fa (Biometria dinamica):** Modelli comportamentali come il riconoscimento vocale, la firma manoscritta o il ritmo di digitazione (keystroke dynamics).

Valutazione del Rischio per l'Autenticazione

Il potenziale impatto di una falla di sicurezza viene classificato per diverse categorie (es. danni finanziari, reputazione, sicurezza personale) in Basso, Moderato e Alto. In base al rischio stimato, si definiscono quattro **Livelli di Garanzia (Assurance Levels)**:

- **Livello 1:** Poca o nessuna fiducia nella validità dell'identità (impatto potenziale basso per tutte le categorie).
- **Livello 2:** Qualche fiducia nell'identità. Utile per interazioni commerciali generiche e mitigazione di rischi moderati.
- **Livello 3:** Alta fiducia. Necessario quando l'impatto potenziale è moderato o alto.
- **Livello 4:** Altissima fiducia. Necessario per proteggere sistemi in cui l'impatto potenziale è sempre alto (es. severe perdite finanziarie, catastrofi o pericoli per la sicurezza personale dell'individuo).

6.2 Autenticazione Basata su Password

Le password sono la linea di difesa più diffusa contro gli intrusi. L'ID utente determina se l'utente è autorizzato ad accedere, stabilisce i privilegi accordati (es. stato di "superuser") ed è impiegato nel controllo degli accessi discrezionale per concedere o negare permessi su file e risorse.

Vulnerabilità delle Password

Le password affrontano diverse minacce, per le quali servono specifiche contromisure:

- **Attacco a dizionario offline:** L'attaccante ottiene il file delle password del sistema e confronta gli hash memorizzati con gli hash generati da un dizionario di password comuni.
- **Attacco a un account specifico:** L'attaccante invia tentativi di password a un account mirato. Si contrasta con un meccanismo di *account lockout* (es. blocco dopo 5 tentativi falliti).

- **Attacco con password popolari:** L'attaccante usa una singola password comunissima e la testa contro una vasta gamma di ID utente, aggirando i blocchi sui singoli account.
- **Indovino mirato:** Sfruttamento di informazioni personali note per dedurre la password della vittima.
- **Dirottamento della workstation (Workstation hijacking):** L'attaccante sfrutta una postazione temporaneamente incustodita ma con un utente loggato. Si previene con il logout automatico per inattività.
- **Errori degli utenti:** Condivisione intenzionale della password, scrittura su appunti cartacei o vulnerabilità all'ingegneria sociale.
- **Sfruttamento di password multiple:** Uso della stessa password su dispositivi di rete diversi. Se un dispositivo debole cade, compromette anche gli altri.
- **Monitoraggio elettronico:** Intercettazione di una password inviata in chiaro sulla rete.

L'Uso di Password Hashate e il "Salt"

Per proteggere il file delle password a riposo, la prassi comune (utilizzata dai sistemi UNIX) è non memorizzare la password in chiaro. Quando viene creata una password, essa è combinata con un **Salt** (un valore casuale di lunghezza fissa). La password e il salt fungono da input per una funzione hash crittografica lenta, che produce un codice hash di lunghezza fissa. Il file memorizza l'ID utente, il Salt *in chiaro* e il codice hash crittografato.

L'inclusione del Salt serve a tre scopi vitali:

1. Impedisce che password duplicate siano visibili nel file: se due utenti scelgono la stessa password, i loro hash differiranno a causa dei salt diversi.
2. Aumenta drasticamente la difficoltà degli attacchi a dizionario offline. Per un salt di b bit, lo spazio delle possibili password da testare per l'attaccante aumenta di un fattore 2^b .
3. Rende quasi impossibile stabilire se una persona con account su più macchine abbia utilizzato la stessa password ovunque.

Le prime implementazioni UNIX usavano *crypt(3)*, basata su una modifica del DES eseguita 25 volte e un salt di 12 bit. Questa implementazione è considerata del tutto inadeguata per l'hardware moderno. Varianti UNIX successive adottano **MD5 crypt** (salt a 48 bit e loop interno da 1000 iterazioni) o il ben più robusto **Bcrypt** (usato in OpenBSD, basato sul cifrario a blocchi Blowfish con un salt da 128 bit). Bcrypt include un parametro di "costo" che aumenta il tempo necessario per eseguire l'hash in base alla potenza di calcolo, tenendo a bada gli aggressori.

Cracking delle Password Scelte dall'Utente

I vecchi attacchi consistevano in variazioni sistematiche di dizionari (ortografie al contrario, aggiunta di numeri o caratteri speciali). Oggi i cracker generano un'enorme

tabella di hash precalcolati nota come **Rainbow Table**, che scambia memoria su disco in favore di incredibili aumenti di velocità, capaci di craccare gran parte delle password in pochi secondi.

Un'altra innovazione è l'analisi probabilistica basata su file di password trapelati nel mondo reale (come i 32 milioni di password di RockYou.com). Utilizzando modelli di Markov e grammatiche libere dal contesto, i software di cracking generano tentativi testando per prime le sequenze di lettere, cifre e simboli matematicamente più comuni, arrivando a scoprire quasi il 40% delle password umane in tempi brevissimi.

Strategie di Selezione delle Password e il Filtro di Bloom

Nascondere il file delle password tramite uno *shadow password file* aiuta, ma non risolve il problema alla radice. La soluzione richiede l'eliminazione di password indovinabili attraverso una verifica proattiva (Complex Password Policy).

Il Filtro di Bloom (Bloom Filter)

I sistemi proattivi controllano le password contro enormi dizionari di parole vietate. Memorizzare e cercare in file da molti Megabyte sarebbe eccessivamente lento e costoso. La soluzione implementata è il **Filtro di Bloom**.

Un filtro di Bloom di ordine k consiste in un insieme di k funzioni hash indipendenti. Si definisce una tabella hash di N bit (inizialmente tutti a 0). Per ogni parola nel dizionario (composto da D parole), si calcolano i k valori hash e si impostano a 1 i bit corrispondenti nella tabella hash.

Quando l'utente inserisce una nuova password, le si applicano le k funzioni. Se *tutti* i bit risultanti sono uguali a 1, la password è rifiutata (si assume faccia parte del dizionario). Questa tecnica comporta una probabilità di "falso positivo" P (rifiuto di una password non presente nel dizionario), approssimabile come:

$$P \approx (1 - e^{-kD/N})^k$$

Scegliendo, ad esempio, $k = 6$ e un rapporto bit-parola pari a 9.6, si può contenere il dizionario in solo 1.2 MB di memoria con un tasso di falsi positivi inferiore all'1%, comprimendo i dati in maniera sbalorditiva senza richiedere ricerca all'interno del dizionario.

6.3 Autenticazione Basata su Token

I token sono oggetti fisici posseduti da un utente ai fini dell'autenticazione. I più diffusi hanno l'aspetto e le dimensioni di carte di credito.

Memory Card e Smart Card

Le **Memory Card** (es. carte con banda magnetica) possono memorizzare dati ma non elaborarli. Vengono usate per l'accesso fisico o combinate con un PIN per gli sportelli Bancomat. Hanno lo svantaggio di richiedere lettori speciali e l'eventuale furto unito alla scoperta del PIN compromette l'intero accesso.

Le **Smart Card** contengono invece un microprocessore completo (con ROM, EEPROM e RAM) e possono possedere circuiti dedicati all'accelerazione crittografica. Comunicano tramite contatti elettrici fisici (Contact) o attraverso antenne a radiofrequenza (Contactless).

I protocolli per l'autenticazione tramite smart token si dividono in tre grandi categorie:

1. **Statico:** L'utente si autentica inserendo un PIN e il token trasmette l'informazione statica al computer, similmente a una memory card.
2. **Generatore di password dinamiche (OTP):** Il token genera periodicamente (es. ogni minuto) una password univoca. Computer e token devono essere sincronizzati per validare l'accesso.
3. **Challenge-Response:** Il computer di destinazione genera una sfida (un numero casuale). Il token elabora una risposta crittografando la sfida attraverso la sua chiave privata inaccessibile, e restituisce la risposta validata.

Carte d'Identità Elettroniche (eID)

Le eID card nazionali fungono da prova d'identità forte. L'esempio della carta di identità tedesca (*neuer Personalausweis*) mostra l'implementazione di funzioni elettroniche separate:

- **ePass:** Accessibile offline solo dalle autorità, legge dati biometrici come volto e impronte digitali attraverso lettori governativi.
- **eID:** Pensato per l'accesso online. Applica il protocollo **PACE (Password Authenticated Connection Establishment)** che assicura che il chip contactless non possa essere letto a distanza senza un esplicito controllo degli accessi; per attivarlo, l'utente deve inserire il PIN o il CAN (Card Access Number) stampato sulla carta.
- **eSign:** Funzione opzionale che immagazzina chiavi private e certificati X.509 generati da Autorità di Certificazione, consentendo la creazione di firme digitali legalmente valide.

6.4 Autenticazione Biometrica

Tenta di autenticare un individuo basandosi sulle sue caratteristiche fisiche (statica) o comportamentali (dinamica) uniche. Il suo costo e la complessità di implementazione sono notevolmente superiori rispetto alle password.

Caratteristiche Utilizzate

Esempi comuni di tratti usati per l'autenticazione sono: **Caratteristiche facciali** (forme geometriche o termogrammi del sistema vascolare del volto tramite infrarossi); **Impronte digitali** (creste e solchi automatizzati); **Geometria della mano**; **Pattern retinico** (fasci venosi unici nel retro dell'occhio scansionati con luce infrarossa debole); **Iride** (struttura dettagliatissima dell'iride); **Firma** (dinamica e pressione della penna); e **Voce** (correlata agli attributi anatomici ma con un certo margine di variazione temporale).

Operazioni del Sistema

Un sistema biometrico esegue prima una fase di *Enrollment* (registrazione) in cui il sensore acquisisce il segnale, un estrattore ne ricava le caratteristiche salienti, e salva queste ultime nel database come *Template* vettoriale numerico dell'individuo.

Successivamente, il sistema può agire in due modi:

- **Verifica (Verification 1:1):** L'utente fornisce una dichiarazione d'identità (tramite PIN o carta). Il sistema biometrico estrae le caratteristiche scansionate al momento e le confronta *solo* col template del database relativo a quell'utente specifico.
- **Identificazione (Identification 1:N):** L'utente si presenta al sensore senza dichiarare l'identità. Il sistema deve scansionare l'intero database per trovare un eventuale match, operazione computazionalmente gravosa.

Accuratezza Biometrica

Poiché le caratteristiche fisiche presentano continue variazioni ambientali, i sistemi non esigono una corrispondenza esatta al 100%, ma calcolano un **Matching Score** (punteggio di corrispondenza). La decisione si basa sull'impostazione di una soglia (threshold) che comporta il bilanciamento di due tassi statistici di errore:

- **False Match Rate (FMR):** La frequenza con cui un impostore viene erroneamente valutato come utente autorizzato (falso positivo).
- **False Nonmatch Rate (FNMR):** La frequenza con cui un utente legittimo viene erroneamente valutato come utente estraneo (falso negativo).

Spostando il valore della soglia, si sacrifica una statistica a vantaggio dell'altra. Abbassare il tasso FMR (più sicurezza) causa l'inevitabile innalzamento del tasso FNMR (minore convenienza per l'utente, che subirà più rigetti errati). Questa dinamica si osserva tracciando la "Operating Characteristic Curve" del sensore.

6.5 Autenticazione Utente Remota

Autenticarsi da remoto tramite reti locali, link o Internet apre la porta ad attacchi fisici di intercettazione e di replay (ripetizione). Le difese si basano sui protocolli di **Challenge-Response** (sfida-risposta).

- **Protocollo per Password:** L'host di destinazione invia una sfida sotto forma di *nonce* (un numero casuale r). L'utente risponde con la quantità $f(r, h(P))$, combinando il nonce con l'hash della password P dell'utente tramite una determinata funzione f . Il server confronta il risultato senza che la password (o il suo hash puro) attraversi la rete. Il nonce difende categoricamente dagli attacchi replay.
- **Protocollo per Token:** Il nonce viene abbinato alla decrittazione di un *passcode* fornito dal token hardware, che viene generato staticamente o in maniera dinamica (OTP).
- **Protocollo Biometrico Statico:** L'utente si scansiona al sensore biometrico. Il sistema lato client crittografa il template derivato, il nonce dell'host e un identifi-

cattore univoco del dispositivo hardware. L'host valuta l'autenticità decifrando il pacchetto e confrontando il template col database.

- **Protocollo Biometrico Dinamico:** Il server richiede l'esecuzione di una sequenza casuale unica (una sfida di parole o lettere da digitare/pronunciare). Il client acquisisce e restituisce il segnale biometrico risultante da quella specifica istruzione, rendendo vani i tentativi degli hacker di usare registrazioni rubate in passato.

6.6 Problemi di Sicurezza per l'Autenticazione Utente

Le minacce all'autenticazione remota coprono un ampio spettro di vettori:

- **Attacchi lato Client:** Ricerca esaustiva o "guessing" mirato nel tentativo di impersonare un utente. Si difende imponendo limitazioni nei tentativi di login e l'uso di entropia elevata.
- **Attacchi all'Host:** Tentativi di furto del database delle password o dei template biometrici dai server centrali. La protezione del database attraverso algoritmi di crittografia e hashing ritardato è essenziale.
- **Eavesdropping e Copia:** Intercettazioni fisiche come il guardare oltre la spalla (Shoulder surfing), l'uso di un keylogger hardware/software, o la clonazione del token/dato biometrico. Mitigati combinando l'approccio con l'autenticazione multi-fattore.
- **Replay:** Inviare un pacchetto di rete precedentemente rubato. Bloccato dai protocolli challenge-response e dai token a generazione casuale (OTP).
- **Trojan Horse:** Software ingannevoli o periferiche alterate (come falsi lettori ATM) installati per mascherarsi da applicazione legittima, catturando le credenziali fornite volontariamente dall'utente.
- **Denial of Service (DoS):** Saturare i server con false autenticazioni. Un'altra tipologia DoS prende di mira in maniera selettiva un singolo utente, provocando appositamente errori multipli di login per costringere il sistema a bloccare (lockout) l'account della vittima.

6.7 Applicazione Pratica: Un Sistema Biometrico dell'Iride

Un eccellente caso applicativo di autenticazione biometrica su scala statale si trova negli Emirati Arabi Uniti (UAE). Per arginare il rischio di rientro illegale di individui precedentemente espulsi e dotati di falsi passaporti, l'UAE decise di adottare un'autenticazione biometrica ad alta scalabilità. Il pattern dell'iride fu scelto in quanto tratto immutabile nel tempo e non correlabile neppure nei gemelli identici.

Tutti gli individui espulsi subiscono un processo di Enrollment (registrazione) in cui una telecamera a infrarossi converte il disegno delle creste dell'iride in un *phase code* compatto (paragonabile al DNA). Ai valichi di frontiera marittimi, aerei e terrestri, il sistema scansiona le iridi di tutti i turisti eseguendo in rete una completa operazione di **Identificazione 1:N** (e non semplice verifica). Tale sistema incrocia i phase code con l'intero database in tempo reale, processando oltre 9 miliardi di confronti giornalieri senza produrre alcun falso positivo noto. Per prevenire le elusioni tramite agenti midriatici

(colliri che dilatano innaturalmente le pupille nascondendo l'iride), il software governativo è stato aggiornato per individuare dilatazioni anomale e allertare i presidi di controllo al confine.

6.8 Caso di Studio: Problemi di Sicurezza per i Sistemi ATM

Nei primi anni 2000, le grandi banche avviarono una massiccia migrazione tecnologica per le macchine Bancomat (ATM), passando dai protocolli OS/2 e dai collegamenti seriali Point-to-Point a moderni sistemi operativi Windows connessi tramite **reti LAN basate su TCP/IP**. Le macchine ATM comunicavano ora con i "Processori" (sistemi centralizzati dedicati all'autorizzazione EFT) attraversando l'intera Intranet bancaria.

Le Vulnerabilità architetturali:

Con l'adozione delle reti TCP/IP, l'unico aggiornamento reale di sicurezza introdotto dalle banche fu la migrazione dal DES al Triple-DES per la sola crittografia del codice PIN. Tutto il resto del pacchetto comunicativo—comprendente i numeri di carta di credito, saldi, scadenze e gli **importi dei prelievi richiesti ed erogati**—veniva instradato nel protocollo TCP/IP in **chiaro**.

Questa topologia causò due categorie catastrofiche di incidenti:

- **Problema di Confidenzialità (Eavesdropping):** Un aggressore infiltrato all'interno della rete bancaria (tramite falle interne o perimetrali) poteva osservare passivamente i pacchetti, incamerando segretamente i dati sensibili in chiaro di milioni di carte di credito, per poi clonarle.
- **Problema di Integrità (Iniezione di Traffico e Spoofing):** Mancando protocolli macchina-macchina di autenticazione e meccanismi anti-replay, un hacker presente nella rete poteva impersonare direttamente il Processore bancario centrale. Generando falsi messaggi TCP di "Operazione Approvata", l'aggressore ordinava all'ATM di sputare fisicamente fuori banconote, senza che il vero server di contabilità venisse mai coinvolto o a conoscenza della transazione in corso.

Le misure di sicurezza richieste per sigillare i bancomat inclusero provvedimenti di segmentazione fisica della rete ATM locale per isolare il traffico da altri dipartimenti bancari, l'applicazione di router con crittografia IPsec a livello network e, in fase definitiva, l'aggiornamento dei software applicativi dell'ATM per garantire l'uso di crittografia End-to-End forte tra il singolo bancomat e il processore, supportata da completi protocolli di Challenge-Response.

Controllo degli Accessi

Se l'autenticazione verifica in modo inequivocabile l'identità dell'utente, il **Controllo degli Accessi (Access Control)** è il meccanismo che determina ciò che quell'identità autenticata può o non può fare all'interno del sistema informatico.

Definizione di Controllo degli Accessi (RFC 4949)

Il processo attraverso il quale l'uso delle risorse del sistema è regolato secondo una politica di sicurezza ed è consentito esclusivamente alle entità autorizzate (utenti, programmi, processi o altri sistemi) in conformità a tale politica.

7.1 Principi del Controllo degli Accessi

Il controllo degli accessi non è una funzione isolata, ma opera in un contesto di sicurezza più ampio composto da tre elementi interconnessi:

- **Autenticazione (Authentication):** La verifica che le credenziali di un'entità siano valide. È il prerequisito assoluto.
- **Autorizzazione (Authorization):** La concessione formale di un diritto o di un permesso a un'entità di sistema per accedere a una risorsa. Determina *chi è fidato per quale scopo*.
- **Audit (Controllo/Revisione):** Una revisione indipendente e l'esame sistematico dei log e delle attività per garantire la conformità alle policy, rilevare violazioni e raccomandare correzioni.

Le Politiche di Controllo degli Accessi

I sistemi informatici implementano una o più categorie fondamentali di policy per governare gli accessi:

1. **Discretionary Access Control (DAC - Discrezionale):** Il controllo si basa sull'identità del richiedente. È "discrezionale" perché un'entità che possiede determinati diritti su una risorsa è libera (a sua discrezione) di trasferire o concedere tali diritti ad altri utenti.
2. **Mandatory Access Control (MAC - Obbligatorio):** Il controllo si basa sul confronto rigoroso tra le *etichette di sicurezza* delle risorse (es. Top Secret, Riservato) e le *clearance* (livelli di autorizzazione) dei soggetti. È "obbligatorio" perché le regole del sistema sovrascrivono la volontà dell'utente: il creatore di un

documento Top Secret non può declassificarlo e condividerlo discrezionalmente con chi non ha il nulla osta adeguato.

3. **Role-Based Access Control (RBAC - Basato sui Ruoli):** Il controllo si basa sulle mansioni aziendali (Ruoli) assegnate agli utenti. L'accesso non è garantito all'individuo in sé, ma al ruolo che sta ricoprendo in quel momento.
4. **Attribute-Based Access Control (ABAC - Basato sugli Attributi):** L'evoluzione più moderna. Valuta dinamicamente regole logiche complesse basate su attributi del soggetto, dell'oggetto e dell'ambiente (es. ora del giorno, geolocalizzazione, livello di minaccia della rete).

7.2 Soggetti, Oggetti e Diritti di Accesso

I tre mattoni fondamentali di qualsiasi architettura di controllo degli accessi sono:

- **Soggetto (Subject):** Un'entità attiva capace di richiedere l'accesso agli oggetti. Generalmente coincide con un *processo* che viene eseguito per conto di un utente o di un'applicazione. I sistemi definiscono tre classi principali di soggetti per facilitare la gestione:
 1. *Owner (Proprietario):* Il creatore della risorsa.
 2. *Group (Gruppo):* Un insieme di utenti nominato, in cui l'appartenenza è sufficiente per ereditare determinati diritti.
 3. *World (Tutti gli altri):* L'insieme degli utenti che hanno accesso al sistema ma non sono né proprietari né membri del gruppo.
- **Oggetto (Object):** Una risorsa passiva che contiene informazioni o fornisce servizi. Esempi includono file, directory, blocchi di memoria RAM, record di database, porte di comunicazione o stampanti.
- **Diritto di Accesso (Access Right):** Il modo specifico in cui un soggetto può interagire con un oggetto. I privilegi classici comprendono: *Read* (lettura e copia), *Write* (scrittura, modifica e cancellazione), *Execute* (esecuzione di codice), *Delete* (rimozione), *Create* (creazione di nuovi elementi) e *Search* (ricerca ed esplorazione delle directory).

7.3 Controllo degli Accessi Discrezionale (DAC)

L'approccio logico alla base del modello DAC è la **Matrice di Accesso (Access Matrix)**, teorizzata negli anni '70 da Lampson, Graham e Denning. È una tabella bidimensionale in cui le righe rappresentano i Soggetti e le colonne rappresentano gli Oggetti. L'intersezione $A[S, X]$ contiene l'insieme dei diritti (es. Read, Write) che il Soggetto S possiede sull'Oggetto X .

Decomposizione della Matrice di Accesso

Poiché i sistemi moderni possiedono migliaia di utenti e milioni di file, mantenere l'intera matrice in memoria è impossibile (sarebbe una matrice gigantesca ed estremamente

"sparsa", cioè piena di celle vuote). I sistemi operativi la scompongono quindi in due strutture dati implementabili:

1. **Decomposizione per colonne (Access Control Lists - ACL):** Ad ogni *Oggetto* viene associata una lista. Questa lista elenca tutti i Soggetti autorizzati e i relativi diritti. È l'approccio standard dei file system moderni perché permette un controllo granulare su una singola risorsa.
2. **Decomposizione per righe (Capability Tickets):** Ad ogni *Soggetto* viene associata una lista (il suo "portafoglio"). Questa lista contiene i "biglietti" (tickets) che certificano a quali oggetti il soggetto può accedere. Per evitare falsificazioni, le capability devono essere crittografate o gestite unicamente dal Kernel del sistema operativo in uno spazio di memoria inaccessibile agli utenti.

Domini di Protezione

Un concetto ancora più flessibile è il **Dominio di Protezione**. Invece di associare i diritti direttamente a un utente, i diritti sono associati a un dominio. Un processo opera all'interno di un dominio e può, se autorizzato, cambiare dominio durante la sua esecuzione (ad esempio, passando da *User Mode* a *Kernel Mode* in risposta a una System Call). Questo implementa il *Principio del Privilegio Minimo*, garantendo che un programma abbia privilegi elevati solo per le specifiche istruzioni che lo richiedono.

7.4 Esempio Pratico: File Access Control in UNIX

Il file system UNIX amministra i file tramite strutture dati chiamate **Inode (Index Node)**. L'inode contiene tutte le informazioni critiche del file (dimensione, proprietario, posizione fisica su disco), escluso il nome del file, che è invece memorizzato nella Directory (che in UNIX è semplicemente un file speciale contenente la mappatura nome-inode).

Modello Tradizionale UNIX a 12 Bit

Il controllo degli accessi in UNIX si basa su un campo di protezione di 12 bit all'interno dell'inode:

- **I primi 9 bit** definiscono i permessi base. Sono divisi in 3 triplette (Read, Write, Execute), rispettivamente per le classi: **Owner** (Utente proprietario), **Group** (Gruppo proprietario) e **Other** (Resto del mondo). Per una directory, il permesso *Read* permette di visualizzare i file contenuti; *Write* permette di creare, rinominare o cancellare file al suo interno; *Execute* permette di attraversare la directory per cercare un file specifico.
- **Gli ultimi 3 bit** specificano comportamenti di sistema speciali:
 1. **SetUID (Set User ID):** Quando questo bit è attivato su un file eseguibile, l'utente che lancia il programma acquisisce temporaneamente i permessi del *proprietario* del file (Effective User ID) invece dei propri (Real User ID). Se il file appartiene al superuser (root), il programma viene eseguito con i massimi privilegi di sistema.

2. **SetGID (Set Group ID):** Simile al SetUID, ma assegna temporaneamente i privilegi del gruppo del file (Effective Group ID). Se applicato a una directory, forza tutti i nuovi file creati al suo interno a ereditare il gruppo della directory, utile per il lavoro di squadra.
3. **Sticky Bit:** Storicamente serviva a mantenere in RAM un eseguibile dopo la chiusura. Oggi, se applicato a una directory pubblica (come `/tmp`), stabilisce che un utente può cancellare o rinominare un file solo se ne è l'effettivo proprietario, indipendentemente dai permessi di Write globali della directory.

Access Control Lists (ACL) Estese

Il modello classico a 12 bit è inflessibile: non permette di concedere l'accesso a un file al solo "Utente B" e all'"Utente C" se non creando un gruppo apposito. Le versioni moderne di UNIX, Linux e MacOS supportano le **Extended ACL**, che permettono di specificare liste dettagliate di utenti e gruppi singoli, superando i limiti storici pur mantenendo la retrocompatibilità (utilizzando i bit del gruppo originale come *Mask* per definire il tetto massimo dei privilegi assegnabili).

7.5 Controllo degli Accessi Basato sui Ruoli (RBAC)

Mentre il DAC lega i permessi all'identità, il **Role-Based Access Control (RBAC)** li lega alle funzioni aziendali (Ruoli). Nelle grandi organizzazioni, il tasso di rotazione del personale (turnover) è altissimo, ma i reparti e le mansioni restano stabili. L'RBAC modella questa realtà con una complessa relazione *Molti-a-Molti*: un utente può detenere più ruoli, e un ruolo (es. "Ingegnere di Produzione") raggruppa molti utenti e fornisce l'accesso a molte risorse specifiche.

I Modelli di Riferimento RBAC (NIST FIPS 140-3)

Il National Institute of Standards and Technology ha standardizzato l'RBAC definendo quattro modelli incrementali:

1. **RBAC0 (Modello Base):** Introduce le quattro entità fondamentali:
 - *Users (Utenti):* L'individuo fisico.
 - *Roles (Ruoli):* La funzione o il titolo (es. Medico, Infermiere).
 - *Permissions (Permessi):* L'approvazione formale per eseguire un'operazione su un oggetto. I permessi vengono assegnati ai ruoli, mai direttamente agli utenti.
 - *Sessions (Sessioni):* Un utente non esegue le operazioni con tutti i suoi ruoli attivati contemporaneamente. Avvia una sessione, durante la quale attiva solo un sottoinsieme di ruoli pertinenti al compito del momento (Principio del Privilegio Minimo).
2. **RBAC1 (Gerarchie di Ruoli):** Introduce l'ereditarietà matematica. Un ruolo di livello superiore eredita automaticamente i permessi dei ruoli subordinati. Ad esempio, il ruolo "Capo Progetto" eredita i permessi dei ruoli "Sviluppatore Senior"

e "Tester", sommandoli ai propri permessi esclusivi. Le gerarchie si rappresentano tramite Grafi Aciclici Diretti (DAG).

3. **RBAC2 (Vincoli / Constraints):** Introduce le regole per imporre politiche aziendali stringenti. Il vincolo più famoso è la **Separazione dei Doveri (Separation of Duties - SoD)**:

- *SoD Statica:* Due ruoli in conflitto non possono mai essere assegnati alla stessa persona. Ad esempio, nessuno può essere contemporaneamente "Cassiere" (che maneggia denaro) e "Revisore dei Conti" (che controlla la contabilità).
- *SoD Dinamica:* Una persona può possedere entrambi i ruoli, ma non può abilitarli contemporaneamente nella stessa *Sessione*.
- *Cardinalità:* Si impone un tetto numerico. Ad esempio, il ruolo "Direttore Generale" può essere assegnato a una sola persona (Cardinalità = 1).

4. **RBAC3 (Modello Consolidato):** Unisce contemporaneamente RBAC1 e RBAC2, inglobando gerarchie e vincoli complessi.

7.6 Controllo degli Accessi Basato sugli Attributi (ABAC)

Con l'esplosione del Cloud Computing e dei Web Services SOA, le regole di business sono diventate troppo fluide per essere modellate solo da rigidi Ruoli. L'**Attribute-Based Access Control (ABAC)** calcola le autorizzazioni valutando un set di attributi attraverso regole logico-booleane dinamiche (Policy).

Le Tre Categorie di Attributi

1. **Attributi del Soggetto:** Identità, età, dipartimento, qualifica o clearance di sicurezza.
2. **Attributi dell'Oggetto (Risorsa):** Classificazione del documento, data di creazione, autore.
3. **Attributi dell'Ambiente (Environment):** Il vero punto di forza di ABAC. L'accesso può dipendere dal contesto: l'ora del giorno (es. login consentito solo tra le 9:00 e le 18:00), l'indirizzo IP di provenienza (solo reti interne), la geolocalizzazione GPS, o il livello di minaccia attuale (es. lockdown automatico in caso di attacco rilevato dall'IDS).

Architettura Logica ABAC

Il sistema ABAC separa la decisione dall'applicazione:

- **Policy Enforcement Point (PEP):** L'interfaccia che riceve la richiesta dell'utente e la blocca finché non ottiene l'autorizzazione.
- **Policy Decision Point (PDP):** Il cervello crittografico. Valuta gli attributi estratti dai database (es. LDAP, HR) confrontandoli con le policy dichiarative standardizzate, comunemente scritte in linguaggio **XACML (eXtensible Access Control Markup Language)**.

7.7 Gestione delle Identità, Credenziali e Accessi (ICAM)

L'**Identity, Credential, and Access Management (ICAM)** è un framework gestionale omnicomprendente. Definisce le politiche per gestire l'intero ciclo di vita di un'identità digitale all'interno dell'impresa:

1. **Identity Management (Gestione delle Identità):** Gestisce la fase di *Provisioning* (creazione dell'account, assegnazione ID all'assunzione) e di *Deprovisioning* (revoca istantanea e globale degli accessi in caso di licenziamento o dimissioni, spesso fallace in assenza di sistemi automatizzati).
2. **Credential Management (Gestione Credenziali):** Il processo di rilascio, rinnovo e revoca degli strumenti fisici e logici di autenticazione, come le Smart Card (es. PIV cards), token OTP o certificati X.509 della PKI aziendale.
3. **Access Management (Gestione degli Accessi):** L'applicazione pratica delle policy RBAC, DAC o ABAC per approvare o negare i login ai server.
4. **Identity Federation (Federazione delle Identità):** Lo strumento per gestire utenti *oltre* i confini fisici aziendali. Permette il **Single Sign-On (SSO)** tra entità separate senza mai trasmettere le password in chiaro.

7.8 Framework di Fiducia (Trust Frameworks)

La Federazione delle Identità prospera grazie ai protocolli crittografici (es. SAML, OpenID Connect), ma necessita di regole legali ferree. L'**Open Identity Trust Framework (OITF)** risolve questo aspetto fornendo un "ecosistema di fiducia".

Invece di negoziare accordi legali individuali tra migliaia di aziende, le società aderiscono a un Trust Framework gestito da terze parti indipendenti (come la Open Identity Exchange - OIX). Il modello OITF disaccoppia i ruoli:

- **Identity Provider (IdP):** Valida fisicamente l'esistenza della persona e ne gestisce l'autenticazione forte.
- **Attribute Provider (AP):** Una terza parte che certifica attributi specifici (es. un'università che certifica l'attributo "Laureato in Ingegneria").
- **Relying Party (RP):** L'applicazione Cloud finale che accetta l'accesso fidandosi ciecamente dei token digitali firmati dall'IdP e dall'AP, sapendo che l'auditing dell'OITF garantisce la robustezza tecnica e legale della filiera.

7.9 Caso di Studio: Sistema RBAC per una Banca

Per comprendere l'importanza critica di RBAC2, si analizza l'architettura dei permessi di un grande istituto di credito (es. Dresdner Bank), dove l'insider threat è pericoloso quanto l'hacking esterno.

Il sistema mappa posizioni ufficiali e funzioni lavorative in ruoli precisi, gestiti in una gerarchia di ereditarietà:

- **Ruolo Cassiere (Clerk):** Possiede permessi strettamente limitati all'interrogazione dei saldi e al trasferimento fondi su sportello.

- **Ruolo Group Manager:** Eredita i poteri del cassiere, ma possiede i diritti aggiuntivi per approvare transazioni superiori a \$50.000 o eseguire storni.
- **Ruolo Revisore (Auditor):** Ha privilegi di lettura (*Read*) su quasi tutti i database per rilevare transazioni anomale, ma è categoricamente privo di permessi di modifica (*Write*).

L'Implementazione della Mutua Esclusività (Separation of Duties):

Il motore RBAC della banca rifiuterà sistematicamente la creazione di un profilo utente che tenti di sommare il ruolo di Cassiere (operatività finanziaria attiva) con quello di Revisore (controllo contabile passivo). Anche qualora il Direttore di Filiale richiedesse questa anomalia per una "carenza di personale", il vincolo matematico bloccherà l'assegnazione, garantendo che nessuna singola persona nel sistema possa materialmente rubare fondi e contestualmente falsificare o cancellare i registri di tracciamento dell'audit che la incriminerebbero.

Programmi e Programmazione: Dalla Teoria alla Vulnerabilità

I programmi informatici costituiscono il cuore pulsante e l'infrastruttura invisibile della società moderna. Al loro livello più basilare, i programmi non sono altro che sequenze di zeri e uni che istruiscono il microprocessore, eppure la loro esecuzione governa sistemi di importanza vitale: dalle reti di telecomunicazione ai mercati finanziari globale, fino alle apparecchiature mediche salvavita. L'enorme e crescente complessità del software contemporaneo, composto spesso da milioni di righe di codice, rende fisicamente impossibile per un singolo sviluppatore, o persino per un intero team di ingegneri, comprendere, tracciare o verificare matematicamente ogni singola istruzione. Di conseguenza, gli utenti e le organizzazioni sono costretti a fare affidamento su software scritto da terze parti, la cui sicurezza e affidabilità intrinseca sono, per natura, incerte.

La sicurezza del software rappresenta quindi una sfida titanica: l'obiettivo non è semplicemente far funzionare un programma in condizioni operative normali e con input corretti (come richiesto dall'ingegneria del software classica), ma garantire che esso resista indefinitamente a condizioni impreviste, a input malformati e, soprattutto, ad attacchi deliberati e maliziosi.

Tassonomia Formale dei Difetti Software

Nell'ingegneria del software e nella sicurezza informatica, è fondamentale utilizzare una terminologia precisa per classificare i problemi:

- **Errore (Error):** Un'azione umana che produce un risultato scorretto (es. il programmatore digita una variabile sbagliata).
- **Bug / Difetto (Fault):** L'implementazione errata a livello di codice causata dall'errore umano (es. un ciclo `while` che non termina correttamente o un controllo dei limiti mancante).
- **Falla Logica (Flaw):** Un difetto a un livello molto più profondo, insito nella progettazione, nell'architettura del sistema o nel protocollo stesso (es. un sistema che trasmette password in chiaro sulla rete).
- **Vulnerabilità (Vulnerability):** Un difetto (bug o flaw) che introduce un rischio di sicurezza specifico, permettendo a un'entità non autorizzata di violare le policy di confidenzialità, integrità o disponibilità del sistema informatico.

8.1 Sviste di Programmazione Non Intenzionali (Nonmalicious Oversights)

La stragrande maggioranza delle vulnerabilità sfruttate dagli hacker non deriva da intenti malevoli originari da parte degli sviluppatori (come l'inserimento deliberato di backdoor), ma da disattenzioni, da vincoli di tempo durante lo sviluppo (time-to-market) e da una mancata comprensione delle complesse interazioni della memoria a basso livello. Questi difetti possono provocare il semplice blocco del programma (Denial of Service accidentale) o, peggio, l'alterazione del flusso di esecuzione a favore di un attaccante.

1. Il Buffer Overflow (Trabocco del Buffer)

Il *buffer overflow* è storicamente la vulnerabilità più perniciosa, studiata e ampiamente sfruttata nella storia dell'informatica (resa celebre dal Morris Worm nel 1988). Un buffer è un blocco contiguo di memoria allocato dal programma per memorizzare temporaneamente dei dati in fase di elaborazione, come una stringa di testo inserita dall'utente o un pacchetto di rete in entrata. Al momento della creazione, il compilatore o il sistema operativo assegna a questo buffer una dimensione fissa e predeterminata (es. 64 byte).

Il difetto si verifica in modo catastrofico quando un programma tenta di scrivere all'interno di questo buffer una quantità di dati superiore alla sua capacità fisica (es. scrivendo 100 byte in un array che ne può contenere solo 64), senza prima aver implementato un rigoroso controllo dei limiti (*bounds checking*). Poiché le operazioni di scrittura in linguaggi a basso livello (come il C o il C++) procedono in modo puramente sequenziale negli indirizzi di memoria, i dati in eccesso "traboccano" (overflow) dai confini del buffer, andando a sovrascrivere silenziosamente le aree di memoria immediatamente adiacenti.

Anatomia di uno Stack Smashing Attack

Il bersaglio principale e più critico del buffer overflow è lo **Stack** (la pila di memoria). Lo stack è una struttura dati LIFO (Last-In, First-Out) utilizzata dinamicamente dai programmi per gestire il flusso delle chiamate a funzione (subroutine). Quando una funzione principale chiama una funzione secondaria, il processore salva nello stack una struttura chiamata *Stack Frame*, che contiene: le variabili locali della funzione, i parametri passati dal chiamante, il Frame Pointer e, aspetto critico per la sicurezza, l'**Indirizzo di Ritorno** (Instruction Pointer / Return Address), ovvero l'indirizzo esatto di memoria a cui il processore deve tornare una volta che la funzione secondaria ha terminato il suo lavoro.

L'Esecuzione dell'Exploit:

1. **Iniezione del Payload:** L'attaccante fornisce un input studiato a tavolino, generalmente composto da tre sezioni: una sequenza di dati fittizi (es. una lunga serie di caratteri 'A') progettata per riempire l'intero buffer legittimo; uno *Shellcode* (un piccolo frammento di codice macchina compilato che esegue comandi malevoli, come aprire un prompt dei comandi); e un *Nuovo Indirizzo di Ritorno* calcolato matematicamente.
2. **La Sovrascrittura:** La funzione vulnerabile (ad esempio la famigerata `strcpy()`, che copia stringhe fino a trovare un carattere terminatore senza curarsi della lunghezza) copia il payload dell'attaccante. I dati traboccano dal buffer locale, distruggono il Frame Pointer e sovrascrivono fisicamente l'Indirizzo di Ritorno legittimo con il Nuovo Indirizzo di Ritorno inserito dall'attaccante.
3. **Il Dirottamento (Hijacking):** Quando la funzione completa il suo compito naturale, esegue l'istruzione di ritorno (es. `RET` in assembly). Il processore legge lo stack, preleva l'indirizzo di ritorno (ora falsificato) e "salta" a quell'indirizzo, che punta esattamente all'inizio dello Shellcode dell'attaccante. Il codice malevolo viene così eseguito nativamente dal processore, ereditando gli stessi privilegi del programma vulnerabile (se il programma era in esecuzione come *root* o Amministratore, il sistema è totalmente compromesso).

2. Incomplete Mediation (Mediazione Incompleta e Validazione dell'Input)

La "mediazione" è il processo metodico attraverso il quale un sistema informatico interviene per verificare la legittimità, l'integrità, il formato e i permessi di una determinata richiesta prima di autorizzarne l'esecuzione. Una falla di *Incomplete Mediation* (Mediazione Incompleta) si manifesta ogni qualvolta un programma accetta dati in input dall'esterno (che sia da un utente umano, da una chiamata API o dalla rete) e li elabora fidandosi della loro correttezza, omettendo una validazione esaustiva.

La regola fondamentale della sicurezza architetturale stabilisce che **qualsiasi dato proveniente dall'esterno del perimetro di fiducia del server deve essere considerato infetto e potenzialmente ostile**. Molti sviluppatori commettono l'ingenuità di affidarsi ai controlli eseguiti sul lato client (come la validazione dei moduli HTML tramite JavaScript nel browser). Tali controlli sono utili esclusivamente per migliorare l'esperienza utente (User Experience), ma offrono sicurezza pari a zero: un attaccante esperto non utilizza il browser legittimo, ma intercetta e forgia direttamente

le richieste HTTP tramite proxy (es. Burp Suite), inviando al server dati completamente manipolati.

Scenari di Mediazione Incompleta:

- **Manipolazione dei Parametri Web (Parameter Tampering):** Consideriamo un portale di e-commerce mal progettato che memorizza il prezzo di un articolo in un campo nascosto di un form o direttamente nella query dell'URL (es. `item=laptop&price=1000`). Un attaccante altera la stringa in `price=1` prima dell'invio. Se il server backend si limita ad acquisire quel valore senza verificarlo incrociandolo col proprio database interno (mancanza di mediazione autonoma), elaborerà una transazione commerciale a un prezzo fraudolento.
- **SQL Injection (Iniezione SQL):** Avviene quando l'input dell'utente viene concatenato direttamente e testualmente all'interno di una query destinata al database, senza alcuna forma di sanitizzazione (escaping) o senza l'uso di query parametrizzate. L'attaccante inserisce metacaratteri e comandi SQL (es. `admin' OR '1'='1`). L'input altera radicalmente l'albero sintattico della query, forzando il database a ignorare la verifica della password e a restituire dati riservati, o persino a eseguire comandi di distruzione delle tabelle (DROP TABLE).
- **Cross-Site Scripting (XSS):** Quando un'applicazione web accetta testo da un utente (es. la sezione commenti di un blog) e lo renderizza sulla pagina di altri utenti senza aver prima neutralizzato o codificato (HTML encoding) i tag eseguibili. L'attaccante inietta script JavaScript (es. `<script>inviaCookie()</script>`). Il server ospita ciecamente lo script, e quando le vittime visitano la pagina, il loro stesso browser esegue il codice malevolo, cedendo il controllo della sessione all'attaccante.

3. Time-of-Check to Time-of-Use (TOCTTOU) e Race Conditions

Le vulnerabilità note come TOCTTOU (Tempo di Controllo contro Tempo di Utilizzo) rappresentano una sottoclasse sofisticata dei problemi di sincronizzazione definiti **Race Conditions** (Condizioni di Corsa o Concorrenza).

Una race condition si verifica in sistemi multitasking e multithreading quando il comportamento finale o lo stato di un programma dipende dall'ordine cronologico imprevedibile in cui flussi di esecuzione concorrenti accedono, leggono o modificano una risorsa condivisa. L'esito corretto del programma si affida erroneamente a tempistiche specifiche che non sono garantite a livello architetturale.

La Dinamica del TOCTTOU:

Il difetto specifico TOCTTOU si palesa quando intercorre un lasso di tempo (anche di pochi millisecondi) tra il momento in cui il sistema operativo verifica rigorosamente una condizione o un permesso di sicurezza (il "Time of Check") e il momento in cui il programma agisce effettivamente sulla risorsa in virtù di quel controllo (il "Time of Use"). Poiché il controllo e l'uso sono istruzioni distinte e *non atomiche* (ovvero possono essere interrotte dallo scheduler del processore a favore di altri processi), si apre una "finestra di vulnerabilità".

Immaginiamo un programma eseguito con privilegi di sistema (SetUID root) che deve scrivere l'output richiesto da un utente in un file temporaneo:

1. **Fase di Controllo (Check):** Il programma verifica se l'utente che lo ha invocato possiede legalmente i permessi di scrittura sul file fornito come parametro (es. `/tmp/utente.txt`). L'esito è positivo.
2. **La Corsa (The Race):** Immediatamente dopo la verifica, il processore sospende momentaneamente il programma per assegnare cicli di clock ad altri processi. L'attaccante, attraverso uno script parallelo ottimizzato, cancella repentinamente il file `/tmp/utente.txt` e crea al suo posto un *link simbolico* (collegamento) avente lo stesso nome esatto, ma che punta direttamente al file critico `/etc/shadow` (che contiene gli hash delle password del sistema operativo).
3. **Fase di Utilizzo (Use):** Il programma originale riprende l'esecuzione. Forte del controllo superato in precedenza, apre il percorso indicato (ignaro che la natura dell'oggetto sottostante sia mutata da file a link) e vi scrive i dati controllati dall'attaccante. Il risultato è la compromissione totale del sistema tramite la corruzione di file nevralgici, operata attraverso un programma legittimo ingannato temporalmente.

Per mitigare le TOCTTOU, l'ingegneria del software impone di evitare i controlli separati dall'uso, privilegiando tecniche in cui si tenta direttamente di aprire la risorsa gestendo le eccezioni (chiedere scusa è meglio che chiedere permesso), oppure impiegando meccanismi di blocco esclusivo (File Locking) a livello kernel per impedire alterazioni durante le transazioni critiche.

8.2 Codice Malevolo: Il Malware (Software Intenzionalmente Dannoso)

In netto contrasto con le sviste e i bug non intenzionali, il **Malware** (crasi di Malicious Software) è codice progettato, scritto e diffuso deliberatamente con l'esplicito intento di violare sistemi, esfiltrare dati sensibili, distruggere infrastrutture o estorcere capitali finanziari.

Mentre un'alta percentuale di malware fa leva su vulnerabilità architetturali ampiamente documentate per le quali i produttori hanno già distribuito patch correttive (sfruttando la negligenza degli utenti che omettono gli aggiornamenti), l'arma più formidabile nell'arsenale cibernetico è l'**Attacco Zero-Day (0-day)**. Uno zero-day è un exploit che colpisce una vulnerabilità sconosciuta al produttore del software e alla comunità di sicurezza nel momento in cui viene sferrato l'attacco. Il nome deriva dal fatto che gli sviluppatori hanno avuto "zero giorni" a disposizione per analizzare il problema e mitigarlo; contro uno zero-day puro, le difese perimetrali basate su firme note sono strutturalmente impotenti.

Tassonomia Avanzata e Ciclo di Vita del Malware

Il panorama delle minacce è frammentato in diverse tipologie, distinte in base ai loro meccanismi di infezione, propagazione e payload (carico utile):

1. I Virus Informatici:

La forma storicamente più antica di malware. Un virus è una porzione di codice parassita che non può esistere in modo autonomo: deve obbligatoriamente agganciarsi e infettare un programma "ospite" legittimo, uno script o un settore di avvio critico (Boot Sector). L'infezione si diffonde unicamente se l'utente umano, o il sistema operativo, esegue involontariamente il file infetto. Il ciclo di vita di un virus complesso si articola in quattro fasi:

- *Fase Dormiente (Dormant Phase)*: Il virus è fisicamente presente ma inattivo, in attesa di una determinata condizione temporale o logica (es. una data specifica, come nel caso del virus Michelangelo).
- *Fase di Propagazione (Propagation Phase)*: Il virus inizia a replicarsi, scansionando i dischi rigidi e inserendo copie esatte, o variate, di sé stesso all'interno di altri eseguibili.
- *Fase di Attivazione (Triggering Phase)*: Il momento in cui si verifica la condizione prestabilita per scatenare l'attacco.
- **Fase di Esecuzione (Execution/Payload)**: Il virus esegue il suo carico utile malevolo, che può variare dalla visualizzazione di un messaggio goliardico alla cancellazione irrevocabile dell'intero File System.

Per sopravvivere agli scanner degli antivirus (che cercano stringhe binarie specifiche dette "firme"), i creatori di virus implementano tecniche di evasione estreme. I virus **Polimorfici** integrano un motore di crittografia interno: a ogni nuova infezione, il corpo del virus viene cifrato con una chiave casuale diversa. L'unica parte visibile è una piccolissima routine di decrittazione (decryptor stub), che a sua volta subisce continue variazioni superficiali. I virus **Metamorfici** si spingono oltre: abbandonano la crittografia in favore della mutazione strutturale. A ogni infezione, il virus analizza e riscrive completamente il proprio codice assembly. Sostituisce i registri della CPU, altera l'ordine delle istruzioni indipendenti e inietta codice fittizio (junk code), risultando in un binario che non ha alcuna somiglianza strutturale con le sue iterazioni precedenti, pur preservando la medesima funzione logica distruttiva.

2. Oltre i Virus: L'Ecosistema Moderno

- **Worm (Vermi)**: Software totalmente autonomi. Al contrario dei virus, i worm non si legano a file ospiti e non necessitano dell'interazione umana per diffondersi. Sondano costantemente le reti alla ricerca di dispositivi esposti con vulnerabilità non patchate o password deboli. Una volta penetrati, usano la macchina infetta per scansionare freneticamente altri nodi, propagandosi a velocità esponenziale e saturando la banda di rete fino a causare un Denial of Service su vasta scala.
- **Trojan Horse (Cavalli di Troia)**: Programmi che si presentano sotto mentite spoglie (es. un videogioco gratuito, un generatore di chiavi per software pirata o un allegato PDF di una falsa fattura). L'utente viene ingannato dalle tecniche di Social Engineering e installa volontariamente il software. Mentre il programma esegue apparentemente la funzione promessa in primo piano, in

background apre silenziosamente backdoor, disabilita i firewall o installa moduli spia (Spyware/Keylogger).

- **Ransomware:** Il malware a scopo di lucro più devastante del decennio. Penetra nei sistemi (spesso tramite email di phishing contenenti Trojan), scansiona le memorie di massa locali e di rete e utilizza algoritmi di crittografia asimmetrica di grado militare per rendere inaccessibili documenti, database e backup. All'utente viene presentata una schermata di estorsione che richiede il pagamento di un riscatto (Ransom), tipicamente in criptovalute come il Bitcoin per garantire l'anonimato del criminale, in cambio della chiave crittografica necessaria al ripristino.
- **Botnet (Reti Zombie):** Reti sommerse composte da centinaia di migliaia di computer, server o dispositivi IoT compromessi. I proprietari ignorano totalmente l'infezione, poiché i "Bot" operano in modo silenzioso. Tutti i nodi infetti sono collegati a server di Comando e Controllo (C&C) gestiti da un'entità criminale detta Botmaster. Le Botnet vengono affittate sul mercato nero del Dark Web per sferrare letali attacchi Distributed Denial of Service (DDoS) contro infrastrutture critiche, per inviare spam su scala planetaria o per effettuare il mining clandestino di criptovalute consumando l'energia elettrica delle ignare vittime.

8.3 Contromisure per Sviluppatori: I Pilastri dell'Ingegneria del Software (Software Engineering)

La sicurezza non è un attributo che può essere "spalmato" a posteriori su un programma difettoso. Affinché il software resista agli attacchi, i principi di sicurezza devono guidare ogni fase dell'Ingegneria del Software, dalla progettazione architetturale (Secure by Design) fino al deployment.

1. Modularità, Coesione e Accoppiamento

Un'applicazione monolitica composta da milioni di linee di codice intrecciate è un incubo matematico e logico: non può essere analizzata, testata o mantenuta in modo sicuro. L'architettura deve essere **Modulare**, ovvero scomposta in componenti gerarchici piccoli, circoscritti e logici.

Affinché la modularità porti benefici di sicurezza reali, deve massimizzare due metriche fondamentali:

1. **Alta Coesione (High Cohesion):** Ogni singolo modulo deve avere uno scopo univoco e focalizzato. Se un modulo "calcola le imposte", non deve contenere al suo interno routine per stampare documenti o gestire connessioni di rete. Moduli altamente coesi sono più facilmente analizzabili e meno prone a effetti collaterali catastrofici in caso di aggiornamenti.
2. **Basso Accoppiamento (Low Coupling):** I moduli devono essere quanto più indipendenti gli uni dagli altri. L'interazione tra moduli deve avvenire esclusivamente tramite interfacce (API) ben documentate e con il minimo passaggio di dati possibile. Un basso accoppiamento assicura che una falla di sicurezza (es. una corruzione della memoria) in un modulo di rendering non si propaghi incontrollatamente, compromettendo il modulo di crittografia.

Strettamente legato a questi concetti è il principio di David Parnas dell'**Information Hiding (Occultamento delle Informazioni)**. Le decisioni di progettazione interne, le strutture dati complesse e lo stato locale di un modulo devono essere nascosti (resi opachi e inaccessibili) al resto del sistema. Questo impedisce ai moduli esterni di alterare lo stato interno aggirando le logiche di controllo e validazione previste dalle interfacce pubbliche.

2. Sospetto Reciproco e Confinamento (Confinement / Sandboxing)

In sistemi distribuiti e complessi, la fiducia cieca è il viatico per l'intrusione. L'architettura deve fondarsi sul principio del **Sospetto Reciproco (Mutual Suspicion)**. Nessun programma dovrebbe dare per scontato che i moduli con cui comunica, anche se scritti dalla medesima azienda, funzionino in modo corretto o siano immuni da compromissioni. Ogni modulo deve operare come un'entità paranoica: deve validare rigorosamente tipi, lunghezze e formati di ogni parametro ricevuto in ingresso e deve proteggere in modo difensivo le proprie strutture dati in uscita.

Quando il sistema operativo o il browser web è costretto a eseguire codice di provenienza incerta o non verificabile (es. applet esterne, plugin o script), entra in gioco il **Confinamento (Sandboxing)**. Il sistema operativo isola fisicamente e logicamente il processo in esecuzione, privandolo dei normali privilegi utente. All'interno della Sandbox, il programma sospetto subisce restrizioni draconiane: non può leggere o scrivere liberamente nel file system, non può inviare pacchetti in rete e non può comunicare con altri processi in esecuzione. Qualora il codice si rivelasse essere un malware aggressivo, la sua capacità distruttiva rimarrebbe arginata e inerte all'interno della prigione virtuale.

3. L'Efficacia e i Limiti Matematici del Software Testing

Il collaudo (Testing) è lo strumento prediletto dall'industria per assicurare la qualità e la sicurezza del software prima del rilascio. Esso si basa sull'esecuzione del programma in un ambiente controllato, immettendo vettori di test e valutandone i risultati. Tuttavia, come celeberramente formulato dal pioniere dell'informatica Edsger W. Dijkstra: **"Il testing può dimostrare inequivocabilmente la presenza di difetti, ma non potrà mai, in alcun caso, dimostrarne la totale assenza"**. Dato il numero praticamente infinito di possibili percorsi di esecuzione e combinazioni di input nel tempo, è computazionalmente impossibile testare un programma in ogni suo singolo stato.

Le due branche metodologiche principali del testing sono:

- **Black-box Testing (Test Funzionale / della Scatola Nera):** Il tester non ha alcuna visibilità né interesse per il codice sorgente o l'architettura interna. Il programma viene trattato come una "scatola chiusa". Si valuta esclusivamente l'interazione tra gli input forniti e gli output restituiti. È essenziale per testare la robustezza contro input inattesi e malformati, e trova la sua massima espressione tecnica nel *Fuzzing* (il bombardamento automatizzato dell'applicazione con enormi volumi di dati pseudocasuali per indurre crash e rivelare buffer overflow).
- **Clear-box Testing (Test Strutturale / White-box):** Il collaudatore ha pieno accesso al codice sorgente e comprende profondamente la logica algoritmica. I

test vengono ingegnerizzati algebricamente per attraversare i meandri del codice, puntando a metriche di *Code Coverage* elevate: si cerca di garantire che ogni singola istruzione (Statement Coverage) e, soprattutto, ogni possibile diramazione logica derivante da cicli e condizionali IF-THEN-ELSE (Branch Coverage) venga eseguita e validata almeno una volta.

4. Progettazione per Contratto (Design by Contract)

Una delle tecniche più robuste per garantire la stabilità strutturale del codice è il *Design by Contract*. Sviluppata originariamente per il linguaggio Eiffel, questa metodologia innalza il livello di rigore stabilendo che il rapporto tra un modulo "Cliente" (chi invoca una funzione) e un modulo "Fornitore" (la funzione stessa) deve essere governato da un contratto formale e inequivocabile, verificato in tempo reale durante l'esecuzione del programma tramite l'uso di *Asserzioni* matematiche.

Il contratto si poggia su tre pilastri:

1. **Precondizioni (Preconditions):** Le clausole obbligatorie che devono essere assolutamente vere prima che la funzione venga eseguita. È responsabilità totale del Cliente fornire input che soddisfino le precondizioni (es. "L'array non deve essere vuoto" o "Il puntatore non deve essere nullo"). Se il Cliente viola la precondizione, la funzione fornitrice si esonera da qualsiasi garanzia e interrompe l'esecuzione (Fail-Safe), proteggendo il sistema da elaborazioni di dati infetti.
2. **Postcondizioni (Postconditions):** Le garanzie irrevocabili offerte dal Fornitore. Se le precondizioni iniziali sono state rispettate, il Fornitore deve garantire che, al termine della sua esecuzione, il sistema si troverà in uno stato specifico (es. "Il nuovo utente è stato registrato nel database in modo univoco"). Se la postcondizione fallisce, è la prova inconfutabile che esiste un bug fatale nella logica interna della funzione fornitrice.
3. **Invarianti (Invariants):** Assiomi globali del sistema o della classe che devono rimanere perennemente veri in ogni stato stabile dell'oggetto (es. "Il saldo del conto corrente sommato al limite di fido deve essere sempre maggiore o uguale a zero"). Le invarianti blindano l'integrità dei dati a lungo termine.

8.4 Conclusioni: La Fallacia della "Security by Obscurity"

Il capitolo si chiude sfatando uno dei miti più deleteri e ricorrenti nella storia dell'ingegneria informatica: il ricorso alla **Security by Obscurity** (Sicurezza tramite Oscurità o Segretezza).

Di fronte alla complessità di mettere in sicurezza i propri sistemi, innumerevoli organizzazioni e sviluppatori si rifugiano nell'illusione che mantenere segreto il codice sorgente, offuscare gli algoritmi logici o impiegare protocolli crittografici "fatti in casa" e non documentati possa tenere lontani gli hacker. Il presupposto errato è che, se gli attaccanti non possono studiare l'interno del sistema, non saranno in grado di individuarne e sfruttarne le falle.

L'esperienza pratica e decenni di studi dimostrano che questo paradigma è disastroso. Gli hacker, spinti da curiosità o lucro, dispongono di strumenti di *Reverse Engineering*

(ingegneria inversa), decompilatori e debugger avanzatissimi che consentono di smontare, analizzare e comprendere l'architettura dei binari eseguiti, riducendo in polvere l'oscurità del codice in poche settimane. Inoltre, la segretezza vieta alla comunità accademica e scientifica di condurre peer-review (revisioni paritetiche) per scoprire e segnalare i difetti sfuggiti ai creatori.

Quando la sicurezza di un sistema si appoggia unicamente sul fatto che le sue specifiche sono segrete, esso non possiede alcuna difesa intrinseca. Nell'istante esatto in cui il segreto viene decodificato, o trapela per colpa di un dipendente negligente (insider threat), l'intero castello difensivo crolla irrimediabilmente. Una sicurezza informatica concreta e duratura deve obbedire al **Principio di Kerckhoffs**: il sistema deve essere progettato in modo da rimanere matematicamente sicuro anche qualora il nemico ne conosca perfettamente l'intero funzionamento interno, l'architettura e il codice sorgente, con l'unica eccezione della segretezza delle chiavi crittografiche. La trasparenza non è una debolezza, ma il fondamento per l'auditing rigoroso e la resilienza architetturale.